



THE COMPLETE QA SECURITY HANDBOOK

The Ultimate Security Testing Interview Guide

Scenario-Based Questions · Practical Testing · Real Company Viva
API Security · OWASP Top 10 · Hands-on Testing

Volume 1 — Security Fundamentals & OWASP Top 10

Edition 2026.1

For Freshers · Junior & Mid-Level SQA · Manual & Automation
API Testers · DevSecOps Beginners

Beginner → Intermediate → Advanced

The Ultimate Security Testing Interview Guide

Scenario-Based Questions · Practical Testing · Real Company Viva · API Security · OWASP Top 10 · Hands-on Testing

The Complete QA Security Handbook

Edition: 2026.1 **Audience:** Freshers · Junior QA · Mid-Level SQA · Manual Testers · Automation Engineers · API Testers · DevSecOps Beginners **Level path:** Beginner → Intermediate → Advanced **Format:** PDF-ready Markdown

Legal & ethical notice. Every technique in this handbook is written for *defensive* security testing — that is, testing systems you own or are explicitly authorised in writing to test. Running these techniques against systems you do not own is a crime in most jurisdictions (e.g. the U.S. Computer Fraud and Abuse Act, the UK Computer Misuse Act 1990, and equivalent laws elsewhere). Always work inside a signed scope / rules-of-engagement document. The labs in Section 13 use intentionally vulnerable applications built for practice.

How to use this handbook

This book is built the way a strong interview candidate should actually think: not memorising payloads, but understanding *why a vulnerability exists, how an attacker exploits it, how a tester proves it, and how a team defends against it*. In interviews the difference between a junior and a senior answer is almost never "knows one more payload" — it is depth of reasoning across those four axes.

Each major topic is written across three depths:

- **Beginner** — the plain-English definition and the one-sentence "why it matters."
- **Intermediate** — architecture, attack flow, and how you test it manually and with tools.
- **Advanced** — edge cases, chained attacks, defence-in-depth, and the senior-level interview framing.

Every chapter follows the **21-part format** you'll see repeated throughout, so you can navigate any chapter the same way:

1. Theory
2. Visual Diagram (ASCII)
3. Real-Life Example
4. How the Attack Happens
5. How the Tester Finds It
6. Manual Test Cases
7. API Test Cases
8. Postman Example
9. Burp Suite Example
10. OWASP ZAP Example
11. Automation Example

- 12. Python Example
- 13. Java Example
- 14. CI/CD Example
- 15. GitHub Actions Example
- 16. Jenkins Example
- 17. Interview Questions
- 18. Scenario-Based Viva
- 19. Expected Answers
- 20. Common Mistakes
- 21. Senior Interview Tips

Note on scope of this volume. This is **Volume 1** of the handbook. It contains the complete front matter, **Section 1 — Security Fundamentals** (fully written) and **Section 2 — OWASP Top 10** (fully written with the 21-part format), plus the master cheat-sheet pack and the quick-revision plans. The remaining sections (API Security deep-dive, 250+ scenario viva, Web/DB/Mobile/Automation, bug reporting, company-style interviews, mock viva banks, and the labs) are structured in the master Table of Contents below and are built as subsequent volumes so that each part is delivered complete rather than as placeholder stubs.

Master Table of Contents

Front Matter - How to use this handbook - The 21-part chapter format - Master glossary (Appendix A)

Section 1 — Security Fundamentals (*in this volume*) - 1.1 The CIA Triad - 1.2 Authentication vs Authorization vs Accounting (AAA) - 1.3 Encryption (symmetric & asymmetric) - 1.4 Hashing, Salt, and Pepper - 1.5 HTTPS, TLS, SSL, and Certificates - 1.6 PKI, Public/Private Keys, and Digital Signatures - 1.7 Sessions, Cookies, and Tokens - 1.8 JWT - 1.9 OAuth 2.0 and OpenID Connect (OIDC) - 1.10 Same-Origin Policy, CORS, and CSRF - 1.11 Security Headers and Content Security Policy - 1.12 Rate Limiting, Reverse Proxy, and WAF - 1.13 Secrets and Key Rotation

Section 2 — OWASP Top 10 (2021) (*in this volume*) - 2.1 A01 Broken Access Control - 2.2 A02 Cryptographic Failures - 2.3 A03 Injection - 2.4 A04 Insecure Design - 2.5 A05 Security Misconfiguration - 2.6 A06 Vulnerable & Outdated Components - 2.7 A07 Identification & Authentication Failures - 2.8 A08 Software & Data Integrity Failures - 2.9 A09 Security Logging & Monitoring Failures - 2.10 A10 Server-Side Request Forgery (SSRF)

Section 3 — API Security (*next volume*) **Section 4 — Scenario-Based API Viva (250+)** (*next volume*) **Section 5 — Web Security Testing** (*next volume*) **Section 6 — Database Security** (*next volume*) **Section 7 — Mobile App Security** (*next volume*) **Section 8 — Automation Security Testing & CI/CD** (*next volume*) **Section 9 — Security Bug Report Writing** (*next volume*) **Section 10 — Real Company-Style Interviews** (*next volume*) **Section 11 — Mock Viva Banks (Beginner/Intermediate/Advanced)** (*next volume*) **Section 12 — Cheat Sheets** (*in this volume*) **Section 13 — Hands-on Labs** (*next volume*) **Section 14 — Final Interview Preparation & Revision Plans** (*in this volume*)

Appendices - Appendix A — Glossary - Appendix B — HTTP status codes reference - Appendix C — References & further reading - Appendix D — Index

Section 1 — Security Fundamentals

Before you can find a vulnerability, you have to know what "secure" is supposed to mean. This section builds the mental model every good security tester carries: what property is being protected, who is allowed to do what, how secrets are protected in transit and at rest, and how identity is proven. If you interview for a QA-security role and cannot explain the CIA triad, the difference between authentication and authorization, or why a JWT with `alg: none` is dangerous, you will not pass the technical screen. Master this section first.

1.1 The CIA Triad

1. Theory

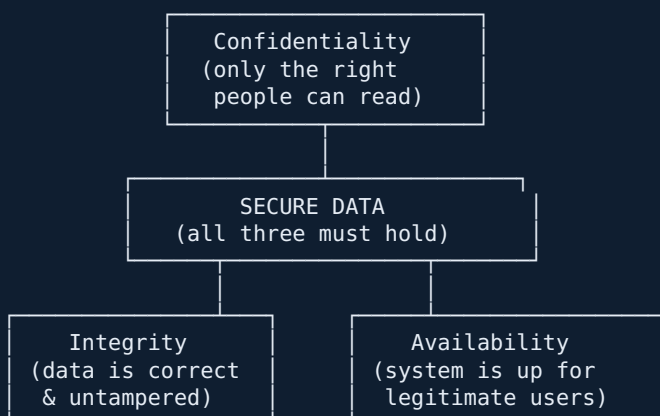
Definition. The CIA triad is the three-part model that defines what information security actually protects: **Confidentiality**, **Integrity**, and **Availability**.

- **Confidentiality** — only authorised parties can read the data. Broken by data leaks, weak encryption, IDOR, over-broad permissions.
- **Integrity** — data cannot be modified by unauthorised parties, and unauthorised modification is detectable. Broken by tampering, injection, missing signatures, race conditions.
- **Availability** — the system is up and responsive for legitimate users. Broken by DoS/DDoS, resource exhaustion, ransomware, or a botched deploy.

Why it exists. Security teams needed a shared vocabulary to reason about *what* an attack costs. Every vulnerability you will ever test maps to at least one of these three. When you write a bug report and rate its severity, you are effectively saying "this breaks Confidentiality/Integrity/Availability to degree X."

Beginner → Intermediate → Advanced. - *Beginner*: keep secrets secret, keep data correct, keep the service running. - *Intermediate*: every OWASP category degrades one or more legs of the triad; know which. - *Advanced*: the triad has tensions — maximum confidentiality (encrypt everything, deny by default) can reduce availability (harder recovery, more failure modes). Real designs trade the three off deliberately. Some models extend it to include **Authenticity**, **Non-repudiation**, and **Accountability** (the "AAA/Parkerian" extensions).

2. Visual Diagram (ASCII)



3. Real-Life Example

A hospital patient-records system. **Confidentiality:** nurse A must not see patient records outside her ward.

Integrity: a medication dosage of "5mg" must never silently become "50mg." **Availability:** during an emergency the system must respond in seconds. A single bug — say an IDOR that lets any logged-in user fetch any patient by incrementing an ID — is a *confidentiality* failure with life-safety consequences.

4. How the Attack Happens

Attackers pick the weakest leg. Confidentiality is attacked with data exfiltration (SQLi dumping a table). Integrity is attacked with tampering (changing a price parameter from `100` to `1`). Availability is attacked with floods or with a single expensive request repeated (an unbounded search that scans the whole DB).

5. How the Tester Finds It

You test each leg explicitly: try to read data you shouldn't (confidentiality), try to change data you shouldn't or in ways the business rules forbid (integrity), and try to make one request that consumes disproportionate resources (availability). A good test plan has cases in all three columns.

6. Manual Test Cases

ID	Leg	Test	Expected
CIA-01	Confidentiality	As user A, request user B's resource by ID	403/404, no data
CIA-02	Integrity	Tamper <code>amount</code> in a POST to a negative value	Rejected, validated server-side
CIA-03	Integrity	Replay a signed request with modified body	Signature check fails
CIA-04	Availability	Send one request with <code>?limit=9999999</code>	Server caps the limit

7. API Test Cases

- Send a request without auth → expect `401`.
- Send with a valid token but for another tenant's object → expect `403/404`.
- Send an oversized payload (e.g. 50 MB JSON) → expect `413 Payload Too Large`, not a crash.

8. Postman Example

```
GET {{base_url}}/api/patients/1002
Authorization: Bearer {{token_userA}}

// Test tab:
pm.test("User A cannot read User B record", () => {
  pm.expect(pm.response.code).to.be.oneOf([403, 404]);
});
```

9. Burp Suite Example

Capture the request to `/api/patients/1001`, send to **Repeater**, change `1001` → `1002`, resend. If you receive another patient's data, confidentiality is broken. Use **Intruder** to sweep a range of IDs and diff response lengths.

10. OWASP ZAP Example

Run **Automated Scan** against the target, then check the **Alerts** tab for "Information Disclosure" and "Access Control" flags. Use the **Access Control Testing** add-on with two user contexts to prove confidentiality boundaries.

11. Automation Example

A regression suite should assert the triad on critical endpoints on every build: an unauthorised read returns 401/403, a tampered write is rejected, and a resource-heavy request is capped.

12. Python Example

```
import requests

BASE = "https://target.example/api"

def test_confidentiality_idor():
    r = requests.get(f"{BASE}/patients/1002",
                    headers={"Authorization": f"Bearer {TOKEN_A}"})
    assert r.status_code in (403, 404), f"IDOR! got {r.status_code}"

def test_integrity_negative_amount():
    r = requests.post(f"{BASE}/transfer",
                    json={"to": "X", "amount": -500},
                    headers={"Authorization": f"Bearer {TOKEN_A}"})
    assert r.status_code == 400
```

13. Java Example

```
import io.restassured.RestAssured;
import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;

public class CiaTests {
    @Test
    public void confidentialityIdor() {
        given().header("Authorization", "Bearer " + TOKEN_A)
            .when().get("/api/patients/1002")
            .then().statusCode(anyOf(is(403), is(404)));
    }
}
```

14. CI/CD Example

Add the triad regression tests to the pipeline as a gate: the build fails if any authorised-access, tampering, or resource-cap assertion regresses.

15. GitHub Actions Example

```
name: security-regression
on: [push, pull_request]
jobs:
  cia-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install requests pytest
      - run: pytest tests/security/test_cia.py -q
```

16. Jenkins Example

```
pipeline {
  agent any
  stages {
    stage('Security Regression') {
      steps { sh 'pytest tests/security/test_cia.py -q' }
    }
  }
}
```

17. Interview Questions

1. What is the CIA triad and which leg does SQL injection primarily break?
2. Give an example where confidentiality and availability are in tension.
3. How would you rate the severity of an IDOR using the triad?

18. Scenario-Based Viva

"An API returns another user's invoices when you change the `id` in the URL. Which part of the triad is broken and how severe is it?"

19. Expected Answers

Confidentiality is broken (unauthorised read). Severity is high because it exposes financial data of arbitrary users; if the same endpoint also accepts writes, integrity is also at risk. A senior candidate adds: severity scales with sensitivity, count of affected records, and whether authentication is required at all.

20. Common Mistakes

- Treating the triad as trivia instead of a severity-scoring lens.
- Forgetting availability (testers often test C and I but never resource exhaustion).
- Conflating "encrypted" with "confidential" — access control failures leak plaintext regardless of TLS.

21. Senior Interview Tips

Frame every finding as "this breaks X leg, costing the business Y." Interviewers remember candidates who reason in impact, not payloads. Mention the extended model (authenticity, non-repudiation) to show depth.

1.2 Authentication vs Authorization vs Accounting (AAA)

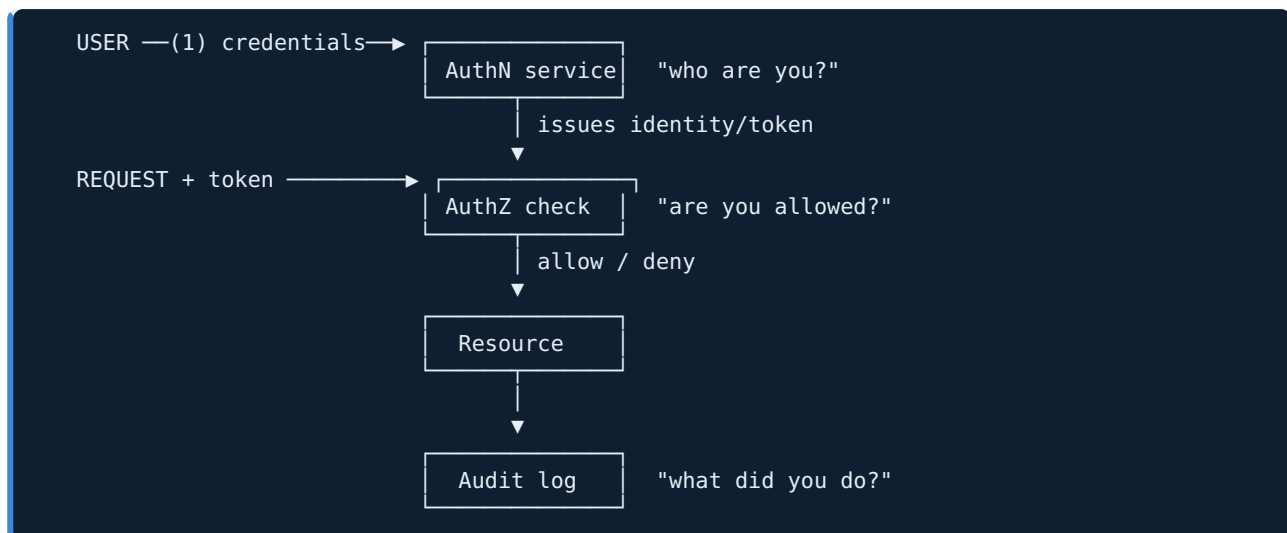
1. Theory

Definition. - **Authentication (AuthN)** — *who are you?* Proving identity (password, token, biometric, certificate). - **Authorization (AuthZ)** — *what are you allowed to do?* Deciding whether an authenticated identity may perform an action on a resource. - **Accounting (Auditing)** — *what did you do?* Recording actions for traceability, billing, and forensics.

Why it exists. Systems must separate "prove yourself" from "here's what you can do." Mixing them is the root of two of the most common vulnerability classes: broken authentication and broken access control. Accounting exists because after an incident you must reconstruct who did what, when.

Beginner → Intermediate → Advanced. - *Beginner:* login = authentication; permissions = authorization; logs = accounting. - *Intermediate:* AuthN produces an identity/token; AuthZ is checked on *every* request, server-side, per resource. - *Advanced:* modern systems use OIDC for AuthN and OAuth scopes / RBAC / ABAC for AuthZ, with a policy engine (e.g. OPA) evaluating attributes. Accounting feeds a SIEM.

2. Visual Diagram (ASCII)



3. Real-Life Example

Online banking. You log in with password + OTP (**AuthN**). You try to view account #999 that isn't yours; the server checks ownership and denies it (**AuthZ**). Every transfer is written to an immutable audit trail (**Accounting**) so fraud can be traced.

4. How the Attack Happens

- Break AuthN: credential stuffing, brute force, weak password reset, session fixation.
- Break AuthZ: IDOR (horizontal), privilege escalation to admin (vertical), forced browsing to hidden endpoints.
- Break Accounting: perform an action the system fails to log, so the attack is invisible.

5. How the Tester Finds It

- AuthN: try no credentials, wrong credentials, locked accounts, token tampering.
- AuthZ: authenticate as a **low-privilege** user and attempt **high-privilege** and **other-user** actions. This is the single highest-value security test in most apps.
- Accounting: perform a sensitive action and confirm a correct, tamper-evident log entry appears.

6. Manual Test Cases

ID	Type	Test	Expected
AAA-01	AuthN	Access protected page with no session	Redirect to login / 401
AAA-02	AuthZ	User role calls admin-only endpoint	403
AAA-03	AuthZ	User A opens User B's object by ID	403/404
AAA-04	Audit	Perform a fund transfer	Log records actor, action, time, result

7. API Test Cases

- Call every endpoint with (a) no token, (b) an expired token, (c) another user's token, (d) a lower-privilege token. Build a matrix of role × endpoint and assert the expected status for each cell.

8. Postman Example

```
POST {{base_url}}/api/admin/users/42/ban
Authorization: Bearer {{token_regular_user}}

pm.test("Regular user cannot ban", () =>
  pm.expect(pm.response.code).to.eql(403));
```

9. Burp Suite Example

Log in as an admin, capture an admin action, then in **Repeater** swap the **Authorization** header for a regular user's token. If the action still succeeds, you have **broken function-level authorization (BFLA)**.

10. OWASP ZAP Example

Define two **Contexts** with two users, run **Access Control Testing**, and let ZAP flag any endpoint reachable by the lower-privilege user that should be admin-only.

11. Automation Example

Maintain a persistent "authorization matrix" test that fails the build if any endpoint becomes reachable by a role that shouldn't reach it.

12. Python Example

```
import pytest, requests
BASE = "https://target.example"
MATRIX = [
    ("/api/admin/users", "regular", 403),
    ("/api/admin/users", "admin", 200),
    ("/api/me", "regular", 200),
]
TOKENS = {"regular": TOKEN_REG, "admin": TOKEN_ADMIN}

@pytest.mark.parametrize("path,role,expected", MATRIX)
def test_authz_matrix(path, role, expected):
    r = requests.get(BASE + path,
                     headers={"Authorization": f"Bearer {TOKENS[role]}"})
    assert r.status_code == expected
```

13. Java Example

```
@ParameterizedTest
@CsvSource({
    "/api/admin/users, regular, 403",
    "/api/admin/users, admin, 200"
})
void authzMatrix(String path, String role, int expected) {
    given().header("Authorization", "Bearer " + token(role))
    .when().get(path)
    .then().statusCode(expected);
}
```

14. CI/CD Example

The authorization matrix runs on every pull request; a new endpoint without an explicit matrix entry fails the pipeline (deny-by-default in tests too).

15. GitHub Actions Example

```
- name: AuthZ matrix
  run: pytest tests/security/test_authz_matrix.py -q
```

16. Jenkins Example

```
stage('AuthZ Matrix') { steps { sh 'pytest tests/security/test_authz_matrix.py' } }
```

17. Interview Questions

1. Difference between authentication and authorization — give an HTTP status code for each failure.
2. Where should authorization be enforced and why not on the client?
3. What is the difference between horizontal and vertical privilege escalation?

18. Scenario-Based Viva

"A mobile app hides the 'Delete User' button for non-admins. Is that secure?"

19. Expected Answers

No — hiding UI is not authorization. The server must reject the delete call regardless of what the client shows. A tester proves this by calling the endpoint directly with a non-admin token. Senior addition: client-side controls are UX, not security; enforce server-side, deny-by-default, and log the attempt.

20. Common Mistakes

- Confusing 401 (not authenticated) with 403 (authenticated but not allowed).
- Testing authorization only through the UI.
- Assuming a valid token means an allowed action.

21. Senior Interview Tips

Say explicitly: "AuthN answers *who*, AuthZ answers *what*, checked server-side on every request, deny-by-default." Then volunteer the horizontal/vertical escalation distinction and mention RBAC vs ABAC. That trio signals real experience.

1.3 Encryption (Symmetric & Asymmetric)

1. Theory

Definition. Encryption transforms readable plaintext into ciphertext using a key, so that only holders of the correct key can recover the plaintext.

- **Symmetric** — the same key encrypts and decrypts (e.g. **AES-256-GCM**). Fast; the challenge is *distributing the shared key securely*.
- **Asymmetric** — a key *pair*: a **public key** (shareable) and a **private key** (secret). What one encrypts, only the other can decrypt (e.g. **RSA, ECC**). Solves key distribution but is slow.
- **Hybrid** (what TLS actually does) — use asymmetric crypto to exchange a random symmetric key, then use fast symmetric crypto for the bulk data.

Why it exists. Confidentiality (and, with authenticated modes, integrity) of data **in transit** and **at rest**. Without encryption, anyone on the network path or with disk access reads everything.

2. Visual Diagram (ASCII)



3. Real-Life Example

When you load `https://bank.example`, TLS uses the bank's certificate (asymmetric) to agree on a per-session symmetric key, then encrypts your whole session with AES (symmetric). At rest, the bank encrypts its database with symmetric keys stored in a KMS/HSM.

4. How the Attack Happens

- Weak/legacy ciphers (DES, RC4), or **ECB mode** which leaks patterns.
- Hard-coded or reused keys/IVs.
- No encryption at rest → a stolen DB dump is plaintext.
- Downgrade attacks forcing old TLS.
- Using encryption without authentication (no MAC) → attacker tampers ciphertext undetected.

5. How the Tester Finds It

Inspect TLS config (ciphers, versions), look for sensitive fields stored in plaintext, check for hard-coded keys in source and mobile binaries, and verify authenticated encryption (GCM/ChaCha20-Poly1305) is used rather than raw CBC/ECB.

6. Manual Test Cases

ID	Test	Expected
ENC-01	Scan TLS with <code>testssl.sh</code>	No SSLv3/TLS1.0, no weak ciphers
ENC-02	Dump a DB backup, grep for PII	Data is ciphertext, not plaintext
ENC-03	Search repo for <code>AKIA</code> , <code>-----BEGIN</code> , <code>password=</code>	No secrets in code

7. API Test Cases

- Confirm the API refuses plain `http://` (redirects to HTTPS or rejects).
- Confirm tokens and PII in responses are not needlessly exposed and are transmitted only over TLS.

8. Postman Example

```
GET http://target.example/api/me // note: http, not https
pm.test("HTTP is rejected/redirected", () =>
  pm.expect(pm.response.code).to.be.oneOf([301, 308, 400, 403]));
```

9. Burp Suite Example

Use Burp's **TLS** view and the target's certificate details; observe negotiated protocol/cipher. Combine with `testssl.sh` for a full cipher audit.

10. OWASP ZAP Example

ZAP passive scan flags "weak SSL/TLS" and "cookie without Secure flag." Review those alerts.

11. Automation Example

Automate a TLS-grade check (e.g. `testssl.sh --jsonfile`) in CI and fail on any weak cipher or protocol.

12. Python Example

```
import ssl, socket
ctx = ssl.create_default_context()
with ctx.wrap_socket(socket.socket(), server_hostname="target.example") as s:
    s.connect(("target.example", 443))
    assert s.version() in ("TLSv1.2", "TLSv1.3"), s.version()
```

13. Java Example

```
SSLSocket sock = (SSLSocket) SSLSocketFactory.getDefault()
    .createSocket("target.example", 443);
sock.startHandshake();
assertTrue(sock.getSession().getProtocol().startsWith("TLSv1.2")
    || sock.getSession().getProtocol().startsWith("TLSv1.3"));
```

14. CI/CD Example

Run `testssl.sh` as a nightly job and gate releases on a passing grade.

15. GitHub Actions Example

```
- name: TLS scan
  run: |
    docker run --rm drwetter/testssl.sh --severity HIGH target.example
```

16. Jenkins Example

```
stage('TLS Scan') {
  steps { sh 'docker run --rm drwetter/testssl.sh target.example' }
}
```

17. Interview Questions

1. Symmetric vs asymmetric — when do you use each?
2. Why does TLS use both?
3. What is wrong with AES-ECB? What is authenticated encryption?

18. Scenario-Based Viva

"A developer says 'we encrypt passwords in the database with AES.' What's your reaction?"

19. Expected Answers

Passwords should be **hashed** (bcrypt/Argon2), not reversibly encrypted. Encryption implies the key exists somewhere to reverse it — if the DB and key leak together, all passwords are exposed. Hashing is one-way by

design. Senior addition: encryption is for data you must read back (PII, card tokens); hashing is for verification-only secrets (passwords).

20. Common Mistakes

- Saying "encrypt passwords" instead of "hash passwords."
- Confusing encoding (Base64) with encryption.
- Forgetting encryption at rest, or forgetting authenticated modes.

21. Senior Interview Tips

Draw the hybrid TLS handshake and name AES-GCM as authenticated encryption. Distinguish encryption (reversible, key-based) from hashing (one-way) from encoding (reversible, no secret). Interviewers love candidates who never confuse those three.

1.4 Hashing, Salt, and Pepper

1. Theory

Definition. A **hash** is a one-way function mapping arbitrary input to a fixed-length digest; you cannot reverse it to recover the input. A **salt** is a unique random value added per record before hashing so identical inputs produce different digests. A **pepper** is a secret value (kept outside the DB, e.g. in an HSM/config) added to all hashes.

- Use **slow, adaptive** password hashes: **Argon2id**, **bcrypt**, or **scrypt** — *never* MD5/SHA-1/SHA-256 alone for passwords.
- Salt defeats **rainbow tables** and stops two users with the same password sharing a digest.
- Pepper adds a secret the attacker won't get from a DB-only breach.

Why it exists. So that a stolen password database does not immediately reveal every password. One-way + slow + salted + peppered = expensive to crack.

2. Visual Diagram (ASCII)

```

password "P@ss"  ┌─┐
                  │ │
per-user salt ───┴─┴─> [ Argon2id( salt + password + pepper ) ] -> digest
(stored w/ hash)
                  ▲
                  |
secret pepper (NOT in the DB)

```

3. Real-Life Example

When a company's user table leaks, the difference between "unsalted MD5" and "Argon2id + salt + pepper" is the difference between all passwords cracked in hours vs a tiny fraction cracked over months. This is why breach severity often hinges on the hashing scheme.

4. How the Attack Happens

Attacker steals the user table, runs a wordlist/rainbow table against fast unsalted hashes, and recovers plaintext passwords, then reuses them elsewhere (credential stuffing).

5. How the Tester Finds It

Review the hashing algorithm and parameters; confirm each user has a unique salt; confirm the algorithm is adaptive (has a cost/work factor). In a code review, flag any use of `md5(`, `sha1(`, or `sha256(` on passwords.

6. Manual Test Cases

ID	Test	Expected
HSH-01	Create two users with the same password; inspect stored hashes	Digests differ (unique salts)
HSH-02	Check algorithm/params	Argon2id/bcrypt/scrypt with sane cost
HSH-03	Attempt to retrieve a password via any endpoint	Impossible; only reset supported

7. API Test Cases

- No endpoint ever returns a password (even hashed).
- Password reset issues a new one-time token; it does not email the old password.

8. Postman Example

```
POST {{base_url}}/api/register {"user":"a","pass":"Same123!"}
POST {{base_url}}/api/register {"user":"b","pass":"Same123!"}
// Then verify in DB (via test hook) the two stored hashes differ.
```

9. Burp Suite Example

Not primarily a Burp task — but use Burp to confirm no endpoint leaks password material and that reset tokens are unpredictable (high entropy) via **Sequencer**.

10. OWASP ZAP Example

ZAP flags weak session/reset token randomness and password-in-URL issues.

11. Automation Example

A unit test asserts that the registration flow produces distinct hashes for equal passwords and that the KDF parameters meet policy.

12. Python Example

```
import bcrypt
def hash_pw(pw: str) -> bytes:
    return bcrypt.hashpw(pw.encode(), bcrypt.gensalt(rounds=12))
def check(pw, stored): return bcrypt.checkpw(pw.encode(), stored)

h1, h2 = hash_pw("Same123!"), hash_pw("Same123!")
assert h1 != h2           # unique salts
assert check("Same123!", h1) # verifies correctly
```

13. Java Example

```
// Spring Security
PasswordEncoder enc = new BCryptPasswordEncoder(12);
String h1 = enc.encode("Same123!");
String h2 = enc.encode("Same123!");
assert !h1.equals(h2);
assert enc.matches("Same123!", h1);
```

14. CI/CD Example

A static analysis rule fails the build if weak hash functions are applied to password variables.

15. GitHub Actions Example

```
- name: Secret & weak-crypto scan
  run: |
    pip install semgrep
    semgrep --config p/secrets --config p/owasp-top-ten --error
```

16. Jenkins Example

```
stage('SAST') { steps { sh 'semgrep --config p/owasp-top-ten --error' } }
```

17. Interview Questions

1. Why hash passwords instead of encrypting them?
2. What does a salt defend against, and why must it be unique per user?
3. What is a pepper and where is it stored?

18. Scenario-Based Viva

"You find passwords stored as SHA-256 with no salt. How bad is it and what do you recommend?"

19. Expected Answers

Bad: SHA-256 is fast and unsalted digests are vulnerable to rainbow tables and GPU cracking; identical passwords share digests. Recommend migrating to Argon2id (or bcrypt) with per-user salts, rehashing on next login. Senior addition: add a pepper in the KMS, enforce a minimum work factor, and plan a forced-reset if a breach is suspected.

20. Common Mistakes

- Calling Base64 "hashing."
- Using SHA-256 for passwords "because it's secure" (it's secure but *fast* — wrong tool).
- Global (shared) salt instead of per-user.

21. Senior Interview Tips

Say "Argon2id, per-user salt, secret pepper, tuned work factor, rehash-on-login for migration." Distinguish encryption vs hashing vs encoding in one breath. That answer alone often clears the crypto portion of a QA-security screen.

1.5 HTTPS, TLS, SSL, and Certificates

1. Theory

HTTPS is HTTP over **TLS**. **SSL** is the obsolete predecessor of TLS (SSL 2/3 are broken and disabled; "SSL certificate" is just common slang for a TLS certificate). TLS provides three things: **encryption** (confidentiality), **integrity** (tamper detection via MAC/AEAD), and **authentication** (you're really talking to the site named in the certificate). A **certificate** binds a public key to a domain and is signed by a **Certificate Authority (CA)** your system trusts. TLS 1.2 and 1.3 are current; 1.3 is faster and drops legacy crypto.

2. Visual Diagram (ASCII) — TLS 1.2 handshake (simplified)

```

Client                                     Server
|----- ClientHello (ciphers, random) ---->|
|<----- ServerHello + Certificate -----|   (cert = pubkey signed by CA)
| [client verifies cert chain]              |
|----- key exchange (encrypt premaster -->|
|         with server pubkey / ECDHE)       |
|<----- Finished (symmetric) -----|
|===== encrypted app data (AES) =====|

```

3. Real-Life Example

Your browser shows a padlock on `https://shop.example` because the site presented a certificate for `shop.example` signed by a CA in your OS/browser trust store, and the handshake produced a shared symmetric key. If an attacker on public Wi-Fi tried to intercept, they could not present a valid certificate for that domain without the CA's private key.

4. How the Attack Happens

- **MITM/downgrade**: force TLS 1.0/weak ciphers; strip HTTPS (SSL-strip) when HSTS is absent.
- **Invalid cert handling**: a client that ignores certificate errors accepts an attacker's cert.
- **Expired/self-signed/wrong-host cert** accepted by a mobile app that disabled validation.
- **Missing HSTS** allows first-request downgrade to HTTP.

5. How the Tester Finds It

Scan TLS versions/ciphers (`testssl.sh` , `ssllscan`), verify HSTS header, verify the app rejects invalid certificates (especially mobile), and confirm `http://` redirects to `https://` .

6. Manual Test Cases

ID	Test	Expected
TLS-01	Request over <code>http://</code>	301/308 to HTTPS
TLS-02	Present expired/self-signed cert to app	Connection refused
TLS-03	Check response headers	<code>Strict-Transport-Security</code> present
TLS-04	<code>testssl.sh</code> target	No SSLv3/TLS1.0, no RC4/DES/EXPORT

7. API Test Cases

Confirm the API endpoint enforces HTTPS, sends HSTS, and (for mobile clients) that certificate pinning is in place where required.

8. Postman Example

```
GET https://target.example/api/health
pm.test("HSTS present", () =>
  pm.expect(pm.response.headers.has("Strict-Transport-Security")).to.be.true);
```

9. Burp Suite Example

Burp acts as a MITM proxy using its own CA; if a **mobile app** trusts Burp's CA without pinning, TLS validation is too weak. Proper pinning breaks Burp interception — that's the *desired* result to verify.

10. OWASP ZAP Example

ZAP passive scan reports missing HSTS, cookies without `Secure` , and weak TLS.

11. Automation Example

Nightly `testssl.sh --jsonfile` parsed by a script that fails on any HIGH finding.

12. Python Example

```
import requests
r = requests.get("https://target.example/", allow_redirects=False)
assert "Strict-Transport-Security" in r.headers
http = requests.get("http://target.example/", allow_redirects=False)
assert http.status_code in (301, 308)
```

13. Java Example

```
given().when().get("https://target.example/")
    .then().header("Strict-Transport-Security", notNullValue());
```

14–16. CI/CD / GitHub Actions / Jenkins

```
# GitHub Actions
- name: TLS grade
  run: docker run --rm drwetter/testssl.sh --severity HIGH target.example
```

```
// Jenkins
stage('TLS') { steps { sh 'ssllscan target.example' } }
```

17. Interview Questions

1. Difference between SSL and TLS? Which versions are safe today?
2. What does a certificate actually prove?
3. What is HSTS and what attack does it prevent?

18–19. Scenario Viva & Expected Answer

"An app works even when the certificate is expired. Is that OK?" — No. Accepting invalid certificates enables MITM; the client must fail closed. For mobile, add certificate pinning. Senior addition: expired cert is an availability/trust failure; pinning defends against rogue CAs but must be paired with a rotation plan to avoid outages.

20. Common Mistakes

- Saying "SSL" and meaning TLS is fine colloquially, but not knowing SSL is deprecated is a red flag.
- Forgetting HSTS; disabling cert validation "to make tests pass."

21. Senior Interview Tips

Sketch the handshake, name ECDHE for forward secrecy, TLS 1.3 for speed, HSTS for downgrade protection, and pinning (with rotation) for mobile.

1.6 PKI, Public/Private Keys, and Digital Signatures

1. Theory

PKI (Public Key Infrastructure) is the system of CAs, certificates, and revocation that lets strangers trust each other's public keys. A **digital signature** is created with a **private key** and verified with the matching **public key**; it proves **authenticity** (who) and **integrity** (unchanged) and provides **non-repudiation** (signer can't deny it). Signing hashes the message, then encrypts the hash with the private key.

2. Visual Diagram (ASCII)

```

SIGNING                                VERIFYING
sender: hash(msg) → encrypt w/ PRIVATE → signature
                                         |
receiver: hash(msg') ==? decrypt(sig w/ sender's PUBLIC)
         if equal → authentic & untampered
  
```

3. Real-Life Example

A software vendor signs its installer; your OS verifies the signature against the vendor's certificate before running it. A JWT is signed by the auth server's private key and verified by APIs with the public key.

4. How the Attack Happens

- Trusting a rogue/compromised CA.
- Not verifying signatures at all ("accept any").
- Using `alg: none` in JWT to skip verification.
- Key theft → attacker signs as you.

5. How the Tester Finds It

Verify that signatures are actually checked, that only expected algorithms are allowed, and that private keys aren't embedded in clients or repos.

6–13. Test cases & code (condensed)

- **Manual:** tamper a signed payload → verification must fail. Swap `alg` to `none` / `HS256` on an `RS256` token → must be rejected.
- **Python:**

```

import jwt # PyJWT
# Rejecting alg:none should raise:
try:
    jwt.decode(tampered, key="", algorithms=["none"])
    assert False, "alg:none accepted!"
except jwt.InvalidAlgorithmError:
    pass
  
```

- **Java (RestAssured):** send a token with modified payload; expect `401`.

14–16. CI/CD

SAST rule to forbid disabling signature verification; secret scanner to catch `-----BEGIN PRIVATE KEY-----` in commits.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** How does a digital signature differ from encryption? (*Signature: sign with private, verify with public, for integrity+authenticity; encryption for confidentiality with the opposite key usage.*)

- **Scenario:** "How would you verify a JWT wasn't tampered with?" — verify the signature with the issuer's public key and enforce the expected algorithm.
- **Mistake:** thinking a signature hides data (it doesn't — it authenticates).
- **Senior tip:** mention non-repudiation, algorithm allow-listing, and key rotation/revocation.

1.7 Sessions, Cookies, and Tokens

1. Theory

HTTP is stateless, so apps track identity with **sessions** (server stores state, client holds a **session ID** in a **cookie**) or **tokens** (client holds a self-contained credential like a **JWT**, often in a header). Cookie security flags matter enormously:

- **HttpOnly** — JavaScript cannot read the cookie (mitigates XSS token theft).
- **Secure** — sent only over HTTPS.
- **SameSite=Lax/Strict** — restricts cross-site sending (mitigates CSRF).
- **Path / Domain** — scope; **_Host-** prefix for locked-down cookies.

2. Visual Diagram (ASCII)

SESSION (stateful) login → server stores state sets Cookie: sid=abc; HttpOnly client sends sid each request server looks up session	TOKEN (stateless) login → server signs a JWT returns token to client client sends Authorization: Bearer JWT server verifies signature, no lookup
--	---

3. Real-Life Example

A banking web app uses server sessions with **HttpOnly; Secure; SameSite=Strict** cookies and rotates the session ID after login (prevents fixation). A mobile app uses short-lived JWT access tokens + refresh tokens.

4. How the Attack Happens

- **Session fixation:** attacker sets a known session ID before login.
- **Session hijacking:** steal the cookie via XSS (if not **HttpOnly**) or network (if not **Secure**).
- **No expiry / no rotation:** stolen sessions live forever.
- **Predictable session IDs.**

5. How the Tester Finds It

Check cookie flags, test that the session ID rotates after login, test idle and absolute timeouts, test logout truly invalidates server-side, and assess ID randomness (Burp Sequencer).

6. Manual Test Cases

ID	Test	Expected
SES-01	Inspect auth cookie flags	<code>HttpOnly; Secure; SameSite</code> set
SES-02	Note session ID pre-login, then login	ID changes (no fixation)
SES-03	Logout, replay old cookie	Rejected server-side
SES-04	Idle N minutes	Session expires

7–13. API / Tools / Code

- **Postman:** assert `Set-Cookie` contains `HttpOnly` and `Secure`.
- **Burp: Sequencer** on session token to measure entropy.
- **Python:**

```
r = requests.post(f"{BASE}/login", json=creds)
sc = r.headers.get("Set-Cookie", "")
assert "HttpOnly" in sc and "Secure" in sc and "SameSite" in sc
```

14–16. CI/CD

DAST (ZAP) in pipeline flags missing cookie flags automatically.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** Sessions vs JWT — trade-offs? (*Sessions: easy revocation, server state; JWT: scalable/stateless but revocation is hard — needs short expiry + denylist.*)
- **Scenario:** "How do you revoke a stolen JWT immediately?" — you can't natively; use short TTLs, a token version/denylist, or rotate signing keys.
- **Mistake:** storing JWTs in `localStorage` (XSS-readable).
- **Senior tip:** prefer `HttpOnly` cookies for browser tokens; explain fixation vs hijacking; mention refresh-token rotation with reuse detection.

1.8 JWT (JSON Web Tokens)

1. Theory

A **JWT** is a signed, base64url-encoded token with three parts: `header.payload.signature`. The header names the algorithm (`alg`), the payload holds **claims** (`sub`, `exp`, `iat`, `iss`, `aud`, roles), and the signature is computed over header+payload with a secret (HS256) or a private key (RS256). **JWTs are signed, not encrypted** — anyone can read the payload; the signature only guarantees integrity/authenticity.

2. Visual Diagram (ASCII)

```

header  {"alg":"RS256","typ":"JWT"}  -base64url
payload {"sub":"42","exp":...,"role":"user"}  —————→ signing-input
signature RS256( signing-input, PRIVATE key ) —————→
token = base64url(header).base64url(payload).base64url(signature)

```

3. Real-Life Example

After login, an auth server returns a JWT. Each microservice verifies the signature with the shared public key and reads `role` / `sub` to authorise — no central session store needed.

4. How the Attack Happens

- **alg: none** — remove the signature; a naive verifier accepts it.
- **Algorithm confusion (RS256 → HS256)** — sign with the *public* key as an HMAC secret; a verifier that allows both is fooled.
- **Weak HMAC secret** — brute-force `HS256` secret offline, then forge tokens.
- **No expiry / no aud / no iss checks** — token replay, cross-service misuse.
- **Sensitive data in payload** — it's readable by anyone.
- **Not verifying at all** — trusting the payload.

5. How the Tester Finds It

Decode the token (jwt.io / `jwt_tool`), try `alg:none`, try RS256 → HS256 confusion, crack weak secrets, tamper claims (`role: admin`), test expired tokens, and test missing/ `null` signature.

6. Manual Test Cases

ID	Test	Expected
JWT-01	Set <code>alg</code> to <code>none</code> , drop signature	Rejected (401)
JWT-02	Change <code>role</code> to <code>admin</code> , keep old sig	Rejected (signature invalid)
JWT-03	Use expired token	Rejected (<code>exp</code> enforced)
JWT-04	RS256→HS256 confusion with public key	Rejected
JWT-05	Brute-force HS256 secret (<code>hashcat -m 16500</code>)	Not crackable (strong secret)

7. API Test Cases

- Reject missing token, malformed token, wrong `aud`, wrong `iss`.
- Reject a valid token from another audience/service.

8. Postman Example

```
GET {{base_url}}/api/me
Authorization: Bearer {{tampered_admin_jwt}}
pm.test("Tampered JWT rejected", () =>
  pm.expect(pm.response.code).to.eql(401));
```

9. Burp Suite Example

Use the **JWT Editor** extension: decode, flip **alg** to **none**, resign with **HS256** using the public key (algorithm confusion), and replay expired tokens — all from Repeater.

10. OWASP ZAP Example

ZAP has a **JWT scan rule** that tests none-alg, weak keys, and signature stripping automatically.

11–13. Automation / Python / Java

```
import jwt, time
# alg:none must be rejected by the server; locally, prove the token is unsigned:
alg_none = jwt.encode({"sub":"42","role":"admin"}, key="", algorithm="none")
# send alg_none to server; expect 401.

# Expired token:
expired = jwt.encode({"sub":"42","exp":int(time.time())-60}, SECRET, algorithm="HS256")
r = requests.get(f"{BASE}/me", headers={"Authorization": f"Bearer {expired}"})
assert r.status_code == 401
```

```
// Java: verify library enforces algorithm allow-list
JWTVerifier v = JWT.require(Algorithm.RSA256(publicKey, null))
    .withIssuer("auth.example").build();
assertThrows(JWTVerificationException.class, () -> v.verify(tamperedToken));
```

14–16. CI/CD

Run **jwt_tool** or a custom suite in the pipeline; SAST rule forbids **algorithms=["none"]** and forbids accepting multiple algorithms.

17. Interview Questions

1. Is a JWT encrypted? Where should you *not* store one?
2. Explain the **alg:none** and RS256 → HS256 attacks.
3. How do you revoke a JWT before it expires?
4. Which claims must you validate server-side?

18. Scenario-Based Viva

"A login API returns a JWT. Walk me through everything you'd test."

19. Expected Answers

Decode and confirm no sensitive data in payload; verify signature is enforced; test `alg:none`, algorithm confusion, weak-secret cracking, tampered `role`, expired `exp`, missing `aud` / `iss`; confirm token sent over HTTPS only and stored in `HttpOnly` cookie (not `localStorage`); test refresh-token rotation and logout/revocation. Senior addition: enforce a strict algorithm allow-list, short access-token TTL, audience/issuer binding, and reuse-detection on refresh tokens.

20. Common Mistakes

- Thinking JWTs are encrypted/confidential.
- Storing JWTs in `localStorage`.
- Accepting multiple algorithms or `none`.
- No `exp`, no `aud` / `iss` validation.

21. Senior Interview Tips

The JWT question is the single most common API-security interview topic. Have the full attack list memorised (`none`, RS256 → HS256, weak secret, tampered claims, expired, missing audience) and always add the defence for each. Mention that "signed ≠ encrypted" unprompted.

1.9 OAuth 2.0 and OpenID Connect (OIDC)

1. Theory

OAuth 2.0 is an **authorization** framework: it lets an app get *delegated access* to resources (via **access tokens** with **scopes**) without handling the user's password. **OIDC** is a thin **authentication** layer on top of OAuth that adds an **ID token** (a JWT proving *who* the user is). Key roles: **Resource Owner** (user), **Client** (app), **Authorization Server** (issues tokens), **Resource Server** (API). The recommended browser/mobile flow is **Authorization Code + PKCE**.

OAuth = "what can this app do on my behalf." OIDC = "who is the user." Interviewers love this one-liner.

2. Visual Diagram (ASCII) — Authorization Code + PKCE

```
User → Client: "Login with Provider"
Client → Auth Server: /authorize?...&code_challenge=CC (S256 of verifier)
User authenticates & consents
Auth Server → Client: redirect ?code=AUTH_CODE (front channel)
Client → Auth Server: /token code=AUTH_CODE&code_verifier=CV (back channel)
Auth Server verifies CV vs CC → returns access_token (+ id_token for OIDC)
Client → Resource Server: Authorization: Bearer access_token
```

3. Real-Life Example

"Sign in with Google" on a third-party app uses OIDC: Google authenticates you and returns an ID token; the app never sees your Google password and only receives the scopes you consented to.

4. How the Attack Happens

- **Missing/loose `redirect_uri` validation** → authorization code stolen to attacker's site.
- **No PKCE** on public clients → intercepted code is replayable.
- **No `state`** → CSRF on the callback.
- **Token leakage** in URLs, logs, or referrer headers.
- **Over-broad scopes**, or accepting an access token where an ID token is required (or vice versa).
- **Implicit flow** (deprecated) leaking tokens in the URL fragment.

5. How the Tester Finds It

Verify exact `redirect_uri` matching, PKCE presence, `state` validation, scope minimisation, token audience checks, and that the ID token signature/issuer/audience/nonce are validated.

6. Manual Test Cases

ID	Test	Expected
OA-01	Tamper <code>redirect_uri</code> to attacker domain	Rejected
OA-02	Omit/alter <code>state</code>	Callback rejected
OA-03	Replay an authorization code	Rejected (single-use)
OA-04	Use access token as ID token	Rejected
OA-05	Request extra scopes	Only consented scopes granted

7–13. API / Tools / Code (condensed)

- **Postman:** use the built-in OAuth 2.0 helper; then manually tamper `redirect_uri` / `state` in raw requests.
- **Burp:** intercept the `/authorize` and `/token` calls; try redirect-uri and code-replay attacks in Repeater.
- **Python:**

```
# redirect_uri tampering must fail
r = requests.get(f"{AUTH}/authorize", params={
    "client_id": CID, "response_type": "code",
    "redirect_uri": "https://attacker.example/cb", "state": "x",
    "code_challenge": CC, "code_challenge_method": "S256"},
    allow_redirects=False)
assert r.status_code in (400, 401, 403)
```

14–16. CI/CD

Contract tests assert redirect-uri allow-list, PKCE requirement, and scope minimisation on each release.

17. Interview Questions

1. OAuth vs OIDC — which is authN, which is authZ?
2. Why PKCE? What attack does `state` prevent?

3. Access token vs ID token vs refresh token?
4. Why is the implicit flow deprecated?

18–19. Scenario & Expected Answer

"A single-page app uses the implicit flow and stores the token in `localStorage`. Concerns?" — Implicit flow leaks tokens in the URL and lacks a back channel; migrate to Authorization Code + PKCE. `localStorage` is XSS-readable; prefer memory or `HttpOnly` cookies. Senior: enforce exact redirect-uri, `state` for CSRF, short-lived access tokens, rotating refresh tokens with reuse detection.

20. Common Mistakes

- Saying OAuth "logs users in" (that's OIDC's ID token).
- Forgetting PKCE and `state`.
- Confusing scopes (authZ) with identity (authN).

21. Senior Interview Tips

Draw the code+PKCE flow, name each role, and volunteer the four token-handling defences (exact redirect-uri, PKCE, state, short TTL). Distinguish access vs ID vs refresh tokens crisply.

1.10 Same-Origin Policy, CORS, and CSRF

1. Theory

Same-Origin Policy (SOP) is the browser rule that script from origin A (scheme+host+port) cannot read responses from origin B. **CORS** is a controlled *relaxation* of SOP: a server uses `Access-Control-Allow-Origin` (and related) headers to opt specific origins into cross-origin reads. **CSRF** is the opposite problem: a malicious site tricks the browser into *sending* a state-changing request to a site where the user is authenticated, riding along the user's cookies.

CORS controls who can **read** your responses. CSRF is about unwanted **writes**. They are different problems; CORS does not fix CSRF and vice versa.

2. Visual Diagram (ASCII)

```
SOP: script@evil.com —fetch→ bank.com/data → browser BLOCKS reading response
CORS: bank.com replies Access-Control-Allow-Origin: https://good.com → good.com may read
CSRF: user logged into bank.com; visits evil.com which auto-submits:
      <form action="bank.com/transfer" method=POST> ... </form>
      browser attaches bank.com cookies → transfer happens if no anti-CSRF token
```

3. Real-Life Example

A bank sets `SameSite=Strict` cookies and requires a per-form CSRF token, so a malicious page cannot forge a transfer. Its API allows `Access-Control-Allow-Origin: https://app.bank.example` only — not `*` — so no arbitrary site can read account data.

4. How the Attack Happens

- **CSRF:** auto-submitting form / image tag to a cookie-authenticated endpoint with no token/SameSite.
- **CORS misconfig:** reflecting the `Origin` header into `Access-Control-Allow-Origin` with `Access-Control-Allow-Credentials: true` → any site reads authenticated responses. Or `ACA0: *` on sensitive data.

5. How the Tester Finds It

- **CORS:** send requests with a crafted/rogue `Origin` header and see if it's reflected with credentials allowed.
- **CSRF:** remove the CSRF token / change `SameSite` behaviour and see if a cross-site state change succeeds.

6. Manual Test Cases

ID	Test	Expected
COR-01	<code>Origin: https://evil.example</code> on API	Not reflected; no <code>ACA0: evil</code>
COR-02	<code>ACA0: *</code> + <code>Allow-Credentials: true</code>	Never both together
CSRF-01	Submit state-change without CSRF token	Rejected
CSRF-02	Cross-site POST with valid cookie	Blocked by SameSite/token

7–13. API / Tools / Code

- **Burp:** add `Origin: https://evil.example`, observe response ACAO header. Use the **CSRF PoC generator** on a state-changing request.
- **Python (CORS probe):**

```
r = requests.get(f"{BASE}/api/account",
                 headers={"Origin": "https://evil.example"})
acao = r.headers.get("Access-Control-Allow-Origin", "")
acac = r.headers.get("Access-Control-Allow-Credentials", "")
assert not (acao == "https://evil.example" and acac == "true"), "CORS misconfig!"
assert acao != "*" or acac != "true"
```

14–16. CI/CD

DAST rule for CORS reflection + a test that any state-changing endpoint requires a CSRF token or is header/token-authenticated (not cookie-only).

17. Interview Questions

1. What is SOP and what does CORS change about it?
2. Does CORS prevent CSRF? Why not?
3. How does `SameSite` mitigate CSRF? Limits?
4. Why is `ACA0: *` with credentials dangerous?

18–19. Scenario & Expected Answer

"An API returns `Access-Control-Allow-Origin: <reflected Origin>` and `Allow-Credentials: true`. Impact?" — Any website can make authenticated cross-origin requests and read the responses, exfiltrating user data — critical. Fix: allow-list specific origins, never reflect arbitrarily, and never pair `*`/reflection with credentials. Senior: note this is *read* exposure; pair with `SameSite` cookies and anti-CSRF tokens for *write* protection.

20. Common Mistakes

- Believing CORS stops CSRF.
- Reflecting `Origin` blindly.
- Cookie-only auth on state-changing endpoints without CSRF tokens.

21. Senior Interview Tips

Say the one-liner: "CORS governs cross-origin *reads*; CSRF is unwanted *writes*." Then give both defences (allow-list + credentials rule for CORS; SameSite + anti-CSRF token / non-cookie auth for CSRF).

1.11 Security Headers and Content Security Policy

1. Theory

Security headers are server-set HTTP response headers that instruct the browser to enable protections:

- **Strict-Transport-Security (HSTS)** — force HTTPS.
- **Content-Security-Policy (CSP)** — allow-list where scripts/styles/frames may load from; the strongest defence-in-depth against XSS.
- **X-Content-Type-Options: nosniff** — stop MIME sniffing.
- **X-Frame-Options** / **CSP frame-ancestors** — clickjacking defence.
- **Referrer-Policy** — limit referrer leakage.
- **Permissions-Policy** — restrict powerful features (camera, geolocation).
- Cache/ **Cross-Origin-*** headers for isolation.

2. Visual Diagram (ASCII)

Response headers → browser behaviour

```
HSTS           → never talk HTTP again to this host
CSP script-src 'self' cdn → only run scripts from self + that CDN (blocks injected <script>)
X-Content-Type-Options nosniff → don't guess MIME (blocks some XSS)
X-Frame-Options DENY / frame-ancestors 'none' → cannot be framed (anti-clickjacking)
```

3. Real-Life Example

A site with `Content-Security-Policy: default-src 'self'; script-src 'self'` neutralises most reflected/stored XSS: even if an attacker injects `<script>`, the browser refuses to execute inline/foreign scripts.

4. How the Attack Happens

Absence enables: XSS (no CSP), clickjacking (no frame-ancestors), protocol downgrade (no HSTS), MIME confusion (no nosniff). A too-permissive CSP (`script-src 'unsafe-inline' *`) is nearly useless.

5. How the Tester Finds It

Inspect response headers on every key page; grade CSP quality (does it allow `'unsafe-inline'`, wildcards?); test framing; test HTTP → HTTPS.

6. Manual Test Cases

ID	Test	Expected
HDR-01	Check CSP header	Present, no <code>'unsafe-inline'</code> for scripts, no <code>*</code>
HDR-02	Try to iframe the page	Blocked (<code>frame-ancestors 'none'</code> / <code>XFO DENY</code>)
HDR-03	Check HSTS	Present with long max-age
HDR-04	Check nosniff	<code>X-Content-Type-Options: nosniff</code>

7–13. API / Tools / Code

- **ZAP/Burp** passive scans list missing headers automatically.
- **Python:**

```
req = {"Content-Security-Policy", "Strict-Transport-Security",
      "X-Content-Type-Options", "X-Frame-Options", "Referrer-Policy"}
r = requests.get(BASE)
missing = req - set(r.headers)
assert not missing, f"Missing headers: {missing}"
assert "unsafe-inline" not in r.headers.get("Content-Security-Policy", "")
```

14–16. CI/CD

Fail the build if required headers are missing or CSP contains `'unsafe-inline'` /wildcards (a simple parser test).

17. Interview Questions

1. Which header most reduces XSS impact, and how?
2. What's wrong with `script-src 'unsafe-inline'`?
3. HSTS vs redirect — why is HSTS stronger?

18–19. Scenario & Expected Answer

"XSS keeps appearing despite input filtering. What defence-in-depth do you recommend?" — A strict CSP (`script-src 'self'` with nonces/hashes, no `'unsafe-inline'`), plus output encoding and `HttpOnly` cookies so stolen XSS can't read the session. Senior: CSP is mitigation not a cure; fix the injection *and* add CSP + Trusted Types.

20. Common Mistakes

- Treating a permissive CSP as protection.
- Setting headers on the homepage but not API/error pages.

21. Senior Interview Tips

Name the "must-have five" (HSTS, CSP, nosniff, frame-ancestors, Referrer-Policy) and explain CSP nonces/hashes vs `'unsafe-inline'`. Mention Trusted Types for DOM XSS.

1.12 Rate Limiting, Reverse Proxy, and WAF

1. Theory

- **Rate limiting** caps how many requests a client/IP/user/key may make in a window — the primary defence against brute force, credential stuffing, enumeration, and cheap DoS.
- **Reverse proxy** (nginx, Envoy, API gateway) sits in front of app servers: TLS termination, routing, caching, and a natural place to enforce rate limits and headers.
- **WAF (Web Application Firewall)** inspects HTTP traffic and blocks known-bad patterns (SQLi/XSS signatures). It's **defence-in-depth**, not a substitute for fixing the code — WAFs can be bypassed with encoding/obfuscation.

2. Visual Diagram (ASCII)

```

Internet → [ WAF ] → [ Reverse Proxy / API Gateway ] → [ App servers ]
           |               | (rate limit, TLS, auth, headers)
           |               | (routing, caching)
           | (signatures)

```

3. Real-Life Example

A login endpoint allows 5 attempts/min/IP and 10/min/account, behind an API gateway that also strips dangerous headers; a WAF blocks obvious SQLi payloads while the app still uses parameterised queries.

4. How the Attack Happens

- No rate limit → unlimited password/OTP guessing, account enumeration, scraping.
- Rate limit only by IP → bypass via rotating IPs / `X-Forwarded-For` spoofing.
- WAF bypass via encoding, case, comments, or chunking.

5. How the Tester Finds It

Fire N rapid requests and observe `429`/lockout; test whether limits key on IP, user, and API key; try `X-Forwarded-For` spoofing; test OTP/reset endpoints specifically.

6. Manual Test Cases

ID	Test	Expected
RL-01	100 logins in 10s	429 /lockout kicks in
RL-02	Spoof X-Forwarded-For per request	Limit still applies (keyed on account too)
RL-03	OTP endpoint rapid guesses	Throttled + attempt cap

7–13. API / Tools / Code

- **Burp Intruder** at high concurrency to observe throttling.
- **Python:**

```
codes = [requests.post(f"{BASE}/login", json=bad).status_code for _ in range(100)]
assert 429 in codes, "No rate limiting on login!"
```

14–16. CI/CD

A load-style test asserts throttling remains in place on auth/OTP endpoints each release.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** Why isn't a WAF enough? (*Bypassable; signatures lag; fix root cause.*)
- **Scenario:** "Login has no lockout. Risk?" — credential stuffing/brute force; add per-account + per-IP limits, exponential backoff, CAPTCHA after threshold, and monitoring.
- **Mistake:** rate-limiting by IP only.
- **Senior tip:** layer limits (IP + account + global), add anomaly detection, and treat WAF as one layer of defence-in-depth.

1.13 Secrets and Key Rotation

1. Theory

Secrets are credentials the app needs but must never expose: API keys, DB passwords, signing keys, tokens. They must live in a **secrets manager / vault** (HashiCorp Vault, AWS Secrets Manager, cloud KMS), be injected at runtime, never committed to source, and be **rotated** on a schedule and immediately after any suspected exposure. **Key rotation** limits the blast radius of a leaked key and is required by many compliance regimes.

2. Visual Diagram (ASCII)

```
code (no secrets) → app → request → [ Vault / KMS ]
                                   | short-lived credential
                                   ↓
                               use, then it expires
rotation: old key → overlap window → new key → retire old key
```

3. Real-Life Example

A CI pipeline pulls a database credential from Vault at deploy time; the credential is dynamic and expires in an hour. Signing keys rotate quarterly with an overlap window so old tokens still verify until they expire.

4. How the Attack Happens

- Secrets committed to Git history (found via scanners like the ones attackers run continuously on public repos).
- Long-lived static keys that never rotate → one leak = permanent access.
- Secrets in env dumps, logs, error messages, or client bundles.

5. How the Tester Finds It

Scan the repo and build artifacts for secrets, check that no secret appears in responses/logs/mobile bundles, verify rotation exists, and confirm least-privilege scoping of each key.

6. Manual Test Cases

ID	Test	Expected
SEC-01	Grep git history for keys/secrets (<code>gitleaks detect</code>)	No secrets in history
SEC-02	Trigger an error	No secret/stack path in response
SEC-03	Decompile mobile app	No embedded keys
SEC-04	Confirm rotation policy	Keys rotate; overlap window exists

7–13. Tools / Code

- Scanners: `gitleaks`, `trufflehog`, `semgrep --config p/secrets`.
- Python (quick heuristic):

```
import re, pathlib
pat = re.compile(r"(AKIA[0-9A-Z]{16}|-----BEGIN [A-Z ]*PRIVATE KEY-----|api[_-]?key\s*=\s*['\"](?:[^\"]|\".\")*)")
hits = [p for p in pathlib.Path(".").rglob("*.py")
        if p.is_file() and pat.search(p.read_text(errors="ignore"))]
assert not hits, f"Secrets found: {hits}"
```

14–16. CI/CD

```
# GitHub Actions – block secrets on every push/PR
- uses: gitleaks/gitleaks-action@v2
```

```
// Jenkins
stage('Secret Scan') { steps { sh 'gitleaks detect --no-git -v' } }
```

17. Interview Questions

1. Where should secrets live, and how are they injected?

2. Why rotate keys, and what's an overlap window?
3. How do you handle a secret already committed to Git?

18–19. Scenario & Expected Answer

"An API key was pushed to a public repo last week. What now?" — Assume compromise: rotate/revoke the key immediately, audit for misuse, purge it from history (BFG/filter-repo) knowing it must still be treated as burned, add a pre-commit secret scanner, and move to a vault with short-lived credentials. Senior: rotation limits blast radius; the fix is prevention (vault + scanning), not just cleanup.

20. Common Mistakes

- Deleting the secret from the latest commit but leaving it in history.
- Rotating without revoking the old key.
- Storing secrets in `.env` files that get committed.

21. Senior Interview Tips

Say "vault + short-lived dynamic credentials + automated rotation + pre-commit and CI secret scanning + least privilege." Treat any exposed secret as permanently compromised and rotate first, investigate second.

Section 1 recap. You now have the vocabulary and mental models: the triad as a severity lens; AAA and the horizontal/vertical escalation distinction; encryption vs hashing vs encoding; TLS/PKI/signatures; sessions vs tokens; the full JWT attack list; OAuth/OIDC roles and flows; the CORS-reads-vs-CSRF-writes distinction; the must-have security headers and CSP; layered rate limiting and WAF-as-defence-in-depth; and secrets management with rotation. Every OWASP category in Section 2 builds on these.

Section 2 — OWASP Top 10 (2021)

The OWASP Top 10 is the industry-standard list of the most critical web application security risks, refreshed periodically by the Open Worldwide Application Security Project. For a QA-security interview it is non-negotiable: you must be able to name all ten, explain each in plain English, describe how you'd *test* for it, and describe the *fix*. This section covers each category using the full 21-part chapter format. Learn the pattern once and every category reads the same way.

Interview framing tip. When asked "what is the OWASP Top 10," never just list names. Say: "It's a risk-ranked awareness list; the current 2021 edition leads with **Broken Access Control** as #1 because it's the most widespread and impactful." Then offer to go deep on any one.

2.1 A01 — Broken Access Control

1. Theory

Definition. Access control enforces *what an authenticated user is allowed to do*. It's **broken** when users can act outside their intended permissions — reading, modifying, or deleting resources that belong to others, or reaching admin functions as a regular user. Two flavours: **horizontal** (accessing another user's data at the same privilege level, e.g. IDOR) and **vertical** (gaining higher privileges, e.g. user → admin).

Why it's #1. It's the most common serious flaw because authorization must be checked *on every request, for every object, server-side* — and developers frequently forget one path, or rely on the UI hiding a button.

2. Visual Diagram (ASCII)

```

user A (id=101) token → GET /api/orders/102   (belongs to user B)
                        |
                        server MUST check: does token owner own order 102?
                        |
        ┌────────── if NO ─────────┐ ┌──────── if check missing ─────────┐
        |                          |                          |
        ▼                          ▼                          ▼
    403 Forbidden                (secure) 200 + user B's order ← BROKEN (IDOR)
  
```

3. Real-Life Example

A ride-sharing app exposes `GET /api/trips/{id}`. Changing the trip ID returns *any* user's trip with pickup/drop-off addresses — a horizontal access-control failure leaking location data of every rider.

4. How the Attack Happens

Attacker authenticates as a normal user, then manipulates identifiers (URL path, query, body, JWT claim), forces browsing to hidden endpoints (`/admin`), or replays a captured admin request with their own token (BFLA). Metadata like `role` in a client-controlled field is trusted by a broken server.

5. How the Tester Finds It

Set up **two users** (and one admin). For each endpoint, try user A accessing user B's objects (horizontal) and user accessing admin functions (vertical). Automate an **authorization matrix** (role × endpoint). This is the highest-value test in most engagements.

6. Manual Test Cases

ID	Test	Expected
BAC-01	User A opens User B's resource by ID	403/404
BAC-02	Regular user calls admin endpoint directly	403
BAC-03	Change <code>role</code> / <code>isAdmin</code> in request body	Ignored server-side
BAC-04	Force-browse <code>/admin</code> , <code>/internal</code>	Not reachable without role
BAC-05	Change owner id in POST body to another user	Rejected

7. API Test Cases

- Every object-fetch endpoint: substitute another tenant's/user's ID → expect denial.
- Every mutating endpoint: attempt with lower-privilege token → expect 403.
- Test at the **function** level (BFLA) and **object** level (BOLA/IDOR) separately.

8. Postman Example

```
GET {{base_url}}/api/orders/{{other_user_order_id}}
Authorization: Bearer {{token_userA}}
pm.test("IDOR blocked", () => pm.expect(pm.response.code).to.be.oneOf([403,404]));
```

9. Burp Suite Example

Capture `GET /api/orders/101` , send to **Intruder**, set the ID as payload position, sweep `1..500` , then **sort by response length/status** — a cluster of `200` s with differing content reveals IDOR. Use **Autorize** extension to auto-test with a low-priv token.

10. OWASP ZAP Example

Configure two **Contexts/users** and run **Access Control Testing**; ZAP flags any resource the low-priv user can reach that policy says they shouldn't.

11. Automation Example

Persist the authorization matrix as data-driven tests; any new endpoint without a matrix entry fails CI (deny-by-default in tests).

12. Python Example

```
import pytest, requests
CASES = [ # (path, token_role, expected)
    ("/api/orders/{B}", "A", (403,404)),
    ("/api/admin/users", "regular", (403,)),
    ("/api/admin/users", "admin", (200,)),
]
@pytest.mark.parametrize("path,role,ok", CASES)
def test_access_control(path, role, ok):
    r = requests.get(BASE+path.format(B=OTHER_ID),
                     headers={"Authorization": f"Bearer {TOKENS[role]}"})
    assert r.status_code in ok
```

13. Java Example

```
@Test void idorBlocked() {
    given().header("Authorization", "Bearer "+tokenA)
    .when().get("/api/orders/"+userBOrderId)
    .then().statusCode(anyOf(is(403), is(404)));
}
```

14–16. CI/CD / GitHub Actions / Jenkins

```
- name: AuthZ matrix
  run: pytest tests/security/test_access_control.py -q
```

```
stage('AuthZ') { steps { sh 'pytest tests/security/test_access_control.py' } }
```

17. Interview Questions

1. What's the difference between horizontal and vertical access-control failures?
2. What is IDOR / BOLA and how do you test for it?
3. Why is hiding a button not access control?
4. Where must authorization be enforced?

18. Scenario-Based Viva

"A support agent can view any customer by changing a customer ID in the URL. Walk me through the finding and fix."

19. Expected Answers

It's a horizontal access-control failure (IDOR/BOLA): the server returns objects without verifying the requester is entitled to them. Fix: enforce ownership/tenant checks server-side on every object access, prefer non-guessable identifiers as defence-in-depth, and add an authorization matrix regression test. Severity high (mass PII exposure). Senior: use centralised policy (RBAC/ABAC), deny-by-default, and log denied attempts.

20. Common Mistakes

- Testing only through the UI.

- Checking authentication but not per-object authorization.
- Assuming UUIDs make objects "safe" (obscurity ≠ access control).

21. Senior Interview Tips

Lead with "deny-by-default, enforced server-side, checked on every object and function." Distinguish BOLA (object) vs BFLA (function). Mention Burp Authorize/ZAP access-control testing to show tooling fluency.

2.2 A02 — Cryptographic Failures

1. Theory

Definition. Failures that expose sensitive data because cryptography is missing, weak, or misused — no encryption in transit/at rest, weak algorithms (MD5, SHA-1, DES, RC4), hard-coded keys, bad randomness, or reversible storage of passwords. (Formerly "Sensitive Data Exposure" — renamed to focus on the *root cause*.)

2. Visual Diagram (ASCII)

```
sensitive data (PII, cards, passwords, tokens)
  in transit → must be TLS 1.2/1.3 (not http, not SSLv3)
  at rest    → must be AES-GCM w/ managed keys (not plaintext, not ECB)
  passwords → must be Argon2id/bcrypt (not MD5/SHA/encrypt/plaintext)
  any of these missing → Cryptographic Failure
```

3. Real-Life Example

A breach dump reveals a `users` table storing passwords as unsalted MD5 and credit-card numbers in plaintext — every account and card is immediately usable. The root cause was cryptographic, not the breach itself.

4. How the Attack Happens

Sniff plaintext on the network, read a stolen DB/backup that's unencrypted, crack fast/unsalted password hashes, or exploit a hard-coded key found in the client to decrypt data.

5. How the Tester Finds It

Audit TLS config; check for sensitive data over HTTP; inspect storage for plaintext PII; review password hashing; grep for hard-coded keys/IVs; verify authenticated encryption modes and secure randomness.

6. Manual Test Cases

ID	Test	Expected
CRY-01	Sensitive endpoint over <code>http://</code>	Refused/redirected
CRY-02	DB backup grep for PAN/PII	Ciphertext only
CRY-03	Password storage review	Argon2id/bcrypt + salt
CRY-04	TLS scan	No weak ciphers/protocols
CRY-05	Card number in response/logs	Masked (e.g. <code>**** 1234</code>)

7–13. API / Tools / Code

- **Tools:** `testssl.sh`, `ssllscan`, `semgrep p/secrets`, `gitleaks`.
- **Python:**

```
r = requests.get(f"{BASE}/api/profile", headers=auth)
body = r.text
assert "password" not in body.lower()
# PAN should be masked, never full 16 digits:
import re
assert not re.search(r"\b\d{16}\b", body), "Unmasked card number!"
```

14–16. CI/CD

SAST for weak crypto + secret scanning + nightly TLS grade gate.

17. Interview Questions

1. Why hash (not encrypt) passwords? Which algorithm?
2. Difference between encoding, encryption, and hashing?
3. What is authenticated encryption and why avoid ECB?

18–19. Scenario & Expected Answer

"An app stores session tokens in the DB in plaintext and serves login over HTTP on internal networks. Risks?" — Tokens are stealable by anyone with DB/network access; internal HTTP is still sniffable (insider/lateral movement). Fix: TLS everywhere, hash/rotate tokens, encrypt at rest with managed keys. Senior: classify data, apply crypto per classification, and centralise key management with rotation.

20. Common Mistakes

- "We use HTTPS so we're fine" (ignores at-rest and misuse).
- Encrypting passwords instead of hashing.
- Confusing Base64 with encryption.

21. Senior Interview Tips

Frame it as *data protection failures*, cover in-transit + at-rest + password-hashing + key-management, and name concrete algorithms (TLS 1.3, AES-GCM, Argon2id).

2.3 A03 — Injection

1. Theory

Definition. Injection occurs when untrusted input is interpreted as *code/commands* by an interpreter — SQL, NoSQL, OS shell, LDAP, XPath, ORM, or a browser (XSS is an injection). The fix is nearly always the same: **separate code from data** using parameterised queries / prepared statements / safe APIs, plus context-aware output encoding for XSS.

2. Visual Diagram (ASCII)

```
VULNERABLE (string concat):
"SELECT * FROM users WHERE name='" + input + "'"
input = ' OR '1'='1' → SELECT ... WHERE name='' OR '1'='1' (returns everything)

SAFE (parameterised):
"SELECT * FROM users WHERE name = ?" , bind(input)
the driver treats input as DATA, never as SQL
```

3. Real-Life Example

A search box builds SQL by concatenation. Entering `' UNION SELECT username,password FROM users--` dumps every credential. The same class of bug in a comment field that renders unescaped becomes stored XSS.

4. How the Attack Happens

Attacker submits payloads that break out of the data context: SQL meta-characters (`' " ; -- UNION`), OS separators (`; | && $(...)`), NoSQL operators (`{"$ne": null}`), or `<script>` for XSS. Blind variants use boolean/time responses to exfiltrate data one bit at a time.

5. How the Tester Finds It

Fuzz every input (params, headers, cookies, JSON, file names) with type-appropriate payloads; look for errors, changed result sets, time delays (blind), or reflected/stored script execution. Use `sqlmap` for SQLi confirmation, Burp/ZAP active scan for breadth.

6. Manual Test Cases

ID	Type	Test	Expected
INJ-01	SQLi	' OR '1'='1 in login	No bypass; parameterised
INJ-02	SQLi (blind time)	1' AND SLEEP(5) --	No delay difference
INJ-03	NoSQL	{"user":{"\$ne":null},"pass":{"\$ne":null}}	Rejected
INJ-04	Cmd	; id / \$(id) in a filename field	No command execution
INJ-05	XSS	<script>alert(1)</script> in a comment	Rendered inert (encoded)

7. API Test Cases

Send injection payloads in JSON bodies, query params, and headers (e.g. `X-Forwarded-For`); confirm parameterisation and that errors don't leak SQL/stack traces.

8. Postman Example

```
POST {{base_url}}/api/search {"q":"' OR '1'='1"}
pm.test("No SQLi bypass", () => {
  pm.expect(pm.response.code).to.be.oneOf([200,400]);
  pm.expect(pm.response.text()).to.not.include("SQL syntax");
});
```

9. Burp Suite Example

Send request to **Intruder** with a SQLi payload list; watch for response-length/time anomalies. Or right-click → **Scan** for active injection checks. For blind, use time-based payloads and Burp's response-time column.

10. OWASP ZAP Example

Active Scan includes SQLi, XSS, command-injection rules; review the **Alerts** with evidence.

11–13. Automation / Python / Java

```
PAYLOADS = ["' OR '1'='1", "; DROP TABLE users;--", "<script>alert(1)</script>",
            "{\"$ne\":null}", "; id", "${id}"]
def test_search_injection():
    for p in PAYLOADS:
        r = requests.post(f"{BASE}/api/search", json={"q": p}, headers=auth)
        assert r.status_code in (200,400)
        assert "SQL" not in r.text and "syntax error" not in r.text.lower()
        assert "<script>alert(1)</script>" not in r.text # reflected XSS check
```

```
// RestAssured: SQLi login must not authenticate
given().contentType(JSON).body(Map.of("user","' OR '1'='1","pass","x"))
.when().post("/api/login").then().statusCode(anyOf(is(400), is(401)));
```

14–16. CI/CD

SAST (semgrep) flags string-built queries; DAST (ZAP baseline) runs on staging each pipeline.

```
- name: ZAP baseline
  uses: zaproxy/action-baseline@v0.12.0
  with: { target: 'https://staging.example' }
```

17. Interview Questions

1. What's the single best defence against SQL injection?
2. Explain blind vs error-based vs union-based SQLi.
3. Is XSS injection? Why? How do stored vs reflected differ?
4. How do you test for NoSQL injection?

18. Scenario-Based Viva

"A login form authenticates when you enter `' OR '1'='1` as the username. What happened and how do you fix it?"

19. Expected Answers

The app concatenates input into SQL, so the payload turns the WHERE clause always-true and returns the first user, bypassing auth. Fix: parameterised queries/prepared statements, least-privilege DB account, input validation as defence-in-depth, and generic error messages. Senior: add WAF as a layer, use an ORM safely, and add regression tests with the payload set.

20. Common Mistakes

- Blacklisting characters instead of parameterising.
- Forgetting injection points beyond forms (headers, JSON, file names).
- Treating XSS separately from "injection."

21. Senior Interview Tips

Say "separate code from data — parameterised queries, safe APIs, context-aware output encoding," then name blind/union/error variants and mention `sqlmap` and least-privilege DB accounts. For XSS, mention CSP + Trusted Types as defence-in-depth.

2.4 A04 — Insecure Design

1. Theory

Definition. Flaws that come from *missing or flawed security design*, not from an implementation bug — the feature works as coded but the design itself is unsafe (no rate limiting on password reset, trusting client-side price, no anti-automation on signup, a workflow that can be bypassed). You cannot patch your way out of insecure design; you must **threat model** and design controls in.

2. Visual Diagram (ASCII)

```
secure coding fixes bugs in a feature
secure DESIGN asks: should this feature exist this way at all?
  e.g. "reset password by answering a public security question" → insecure by design
  fix at design time via THREAT MODELLING (STRIDE): Spoof/Tamper/Repudiate/Info/DoS/Elevate
```

3. Real-Life Example

An e-commerce checkout trusts a `price` field sent from the browser. No code is "buggy," but the *design* lets a user buy a laptop for \$1 by editing the request. The fix is architectural: derive price server-side from the product catalogue.

4. How the Attack Happens

Abuse legitimate features: bulk-create accounts (no anti-automation), reset any password (weak recovery design), replay a coupon infinitely (no server-side idempotency/limit), or bypass a multi-step workflow by jumping to the final step.

5. How the Tester Finds It

Threat-model the feature: enumerate abuse cases and trust boundaries. Test business logic: negative quantities, price/coupon tampering, workflow step-skipping, and missing anti-automation on sensitive actions.

6. Manual Test Cases

ID	Test	Expected
DES-01	Tamper <code>price</code> / <code>total</code> client-side	Server recomputes; rejected
DES-02	Apply same coupon 100×	Enforced single use
DES-03	Negative quantity / negative amount	Rejected
DES-04	Skip to final checkout step directly	Blocked; state validated
DES-05	Mass-signup automation	Rate limit/CAPTCHA triggers

7–13. API / Tools / Code

- Business-logic tests are mostly manual + scripted assertions rather than scanner-found.

```
# price tampering must fail
r = requests.post(f"{BASE}/api/checkout",
                  json={"item": "laptop", "price": 1, "qty": 1}, headers=auth)
assert r.status_code == 400 or r.json()["charged"] != 1
```

14–16. CI/CD

Encode key business rules (single-use coupon, server-side pricing, quantity bounds) as regression tests that run every pipeline.

17. Interview Questions

1. What distinguishes insecure design from a security bug?
2. What is threat modelling? Name a framework (STRIDE).
3. Give an example of a business-logic vulnerability.

18–19. Scenario & Expected Answer

"A user gets unlimited free shipping by re-applying a promo code. Bug or design flaw?" — Design flaw: the promo lacks server-side single-use enforcement and idempotency. Fix in design: bind promo to account/order, enforce usage limits server-side, and add abuse monitoring. Senior: catch this with threat modelling and abuse-case testing, not just functional tests.

20. Common Mistakes

- Assuming a scanner will find logic flaws (it won't).
- Trusting client-supplied values (price, role, totals).
- Skipping threat modelling.

21. Senior Interview Tips

Talk about **abuse cases** and **threat modelling (STRIDE)**, and give the classic "trust the client price" example. This category separates candidates who think like attackers from those who only run tools.

2.5 A05 — Security Misconfiguration

1. Theory

Definition. The system is insecure because it's *configured* insecurely: default credentials, unnecessary features/ports/services enabled, verbose error messages/stack traces, directory listing, missing security headers, permissive cloud storage buckets, debug mode in production, or an unpatched default install.

2. Visual Diagram (ASCII)

```
defaults left on ┌
debug=true in prod ┤ ── Security Misconfiguration
stack traces shown ┤
open S3 bucket ───┘
admin/admin login ┌
fix: hardened, minimal, repeatable config (IaC + scanning)
```

3. Real-Life Example

A cloud storage bucket is world-readable, exposing millions of records; or an app runs with `DEBUG=true`, printing full stack traces and secrets on error. Neither is a code bug — it's configuration.

4. How the Attack Happens

Log in with vendor defaults, read secrets from verbose errors, list directories, access an exposed admin console, or download an entire public bucket. Attackers scan the internet continuously for these.

5. How the Tester Finds It

Check for default creds, debug mode, verbose errors, directory listing, exposed management endpoints/ports, missing headers, and open cloud storage. Use configuration scanners and header checks.

6. Manual Test Cases

ID	Test	Expected
MIS-01	Try <code>admin/admin</code> , vendor defaults	Rejected
MIS-02	Trigger a 500 error	Generic message, no stack trace
MIS-03	Browse <code>/uploads/</code>	No directory listing
MIS-04	Access <code>/actuator</code> , <code>/console</code> , <code>.git/</code>	Not exposed
MIS-05	Header scan	Security headers present

7–13. API / Tools / Code

- **Tools:** Nikto, `nuclei`, ZAP passive scan, cloud config scanners.

```
for path in ["/.git/config","/.env","/actuator/env","/console"]:
    r = requests.get(BASE+path)
    assert r.status_code in (401,403,404), f"Exposed: {path}"
```

14–16. CI/CD

Scan container images and IaC (`trivy`, `checkov`); fail on high-severity misconfig. Enforce config via Infrastructure-as-Code so environments are reproducible.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** Give three common misconfigurations. (*Defaults, debug in prod, verbose errors / open buckets / missing headers.*)
- **Scenario:** "Production returns full stack traces on error." — Info disclosure aiding attackers; disable debug, return generic errors, log details server-side, add error-handling tests.
- **Mistake:** shipping dev config to prod; leaving `.git` / `.env` reachable.
- **Senior tip:** advocate hardened baselines, IaC, image scanning, and least functionality ("turn everything off, enable only what's needed").

2.6 A06 — Vulnerable and Outdated Components

1. Theory

Definition. Using libraries, frameworks, runtimes, or OS packages with known vulnerabilities (CVEs), or that are unmaintained/outdated. Modern apps are mostly third-party code, so a single vulnerable dependency (e.g. a Log4j-style flaw) can compromise the whole app. Defence = **know your inventory (SBOM)**, **scan continuously (SCA)**, and **patch fast**.

2. Visual Diagram (ASCII)

```

your code (10%) — depends on → libraries (90%) — depend on → transitive deps
                        |
                        one has a known CVE
                        ▼
                    exploit ships to your app for free
defence: SBOM + SCA (dependabot/snyk/trivy) + fast patching

```

3. Real-Life Example

An app pins an old JSON or logging library with a remote-code-execution CVE. Attackers scan the internet for that version and exploit it en masse — the app never wrote a line of vulnerable code itself.

4. How the Attack Happens

Attacker fingerprints versions (headers, error pages, JS bundles), matches to public CVEs/exploits, and runs the known exploit. Supply-chain attacks add malicious code into a popular package.

5. How the Tester Finds It

Generate/inspect the dependency tree, run **Software Composition Analysis** (OWASP Dependency-Check, Snyk, Trivy, `npm audit`, `pip-audit`), and check runtime/OS versions. Fingerprint exposed versions and cross-reference the NVD.

6. Manual Test Cases

ID	Test	Expected
CMP-01	Run SCA on dependencies	No known-critical CVEs
CMP-02	Check server/framework banners	Versions not needlessly disclosed
CMP-03	Confirm patch process/SLA exists	Documented, timely

7–13. Tools / Code

```

# GitHub Actions — dependency + container scan
- run: pip install pip-audit && pip-audit
- uses: aquasecurity/trivy-action@master
  with: { image-ref: 'myapp:latest', severity: 'CRITICAL,HIGH', exit-code: '1' }

```

Also enable **Dependabot**/Renovate for automated update PRs.

14–16. CI/CD

Fail the build on CRITICAL/HIGH CVEs; require an SBOM (**syft**) artifact per release; auto-open update PRs.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** How do you know if your app uses a vulnerable library? (*SBOM + SCA + NVD matching in CI.*)
- **Scenario:** "A critical CVE drops for a library you use. Process?" — assess exposure/reachability, patch or mitigate, deploy, verify, communicate; add a scanning gate to catch it earlier next time.
- **Mistake:** scanning once instead of continuously; ignoring transitive deps.
- **Senior tip:** emphasise SBOM, continuous SCA, reachability analysis, and a patch SLA. Name Dependency-Check/Snyk/Trivy.

2.7 A07 — Identification and Authentication Failures

1. Theory

Definition. Weaknesses in confirming identity: weak/there's-no lockout password policies, credential stuffing/brute force, weak or leaked-password acceptance, broken "forgot password," missing MFA, session fixation, exposed session IDs, and predictable tokens. (Formerly "Broken Authentication.")

2. Visual Diagram (ASCII)

```
login → [no rate limit] → brute force / credential stuffing succeeds
reset → [predictable token / user enumeration] → account takeover
session → [no rotation after login] → fixation
defence: MFA + lockout/backoff + strong reset tokens + session rotation + breached-pw check
```

3. Real-Life Example

A login with no lockout and no MFA is hit by credential stuffing using passwords from other breaches; thousands of accounts are taken over overnight. A "forgot password" that says "no such user" also enables account enumeration.

4. How the Attack Happens

Brute force / credential stuffing (no throttling), enumerate valid users via different responses/timing, hijack via session fixation, or abuse a weak reset flow (guessable token, token in URL/logs, no expiry).

5. How the Tester Finds It

Test lockout/backoff, MFA enforcement, generic error messages (no user enumeration), reset-token entropy/expiry/single-use, session-ID rotation after login, and logout invalidation. Use Burp Sequencer for token randomness.

6. Manual Test Cases

ID	Test	Expected
AUTH-01	20 bad logins	Lockout/backoff/CAPTCHA
AUTH-02	Valid vs invalid username errors	Identical (no enumeration)
AUTH-03	Reset token: reuse / expired / another user's	Rejected
AUTH-04	Session ID before vs after login	Rotated
AUTH-05	MFA required for sensitive action	Enforced

7–13. API / Tools / Code

```
codes = [requests.post(f"{BASE}/login",
                      json={"u":"victim","p":f"guess{i}"}).status_code for i in range(30)]
assert 429 in codes or 423 in codes, "No brute-force protection"
# user enumeration: responses should not differ by existence
a = requests.post(f"{BASE}/login", json={"u":"real@x.com","p":"x"}).text
b = requests.post(f"{BASE}/login", json={"u":"nope@x.com","p":"x"}).text
assert a == b, "User enumeration via differing responses"
```

14–16. CI/CD

Regression tests assert lockout, generic errors, token expiry, and session rotation remain intact each release.

17. Interview Questions

1. How do you defend a login against brute force and credential stuffing?
2. What is user enumeration and how do you prevent it?
3. What makes a password-reset flow secure?
4. Session fixation vs hijacking?

18–19. Scenario & Expected Answer

"Forgot-password emails a 4-digit code valid for 24 hours with unlimited attempts. Problems?" — Tiny keyspace + long validity + no attempt cap = trivially brute-forced account takeover. Fix: high-entropy single-use token, short expiry, attempt throttling, and no user enumeration. Senior: add MFA, breached-password checks, and monitoring/alerting on reset abuse.

20. Common Mistakes

- No lockout/backoff; different messages for valid vs invalid users.
- Reset tokens with low entropy, long life, or reuse.
- Not rotating session IDs after login.

21. Senior Interview Tips

Cover the full lifecycle: registration, login (MFA + throttling + breached-password check), session (rotation, timeouts, secure cookies), and recovery (strong single-use tokens). Mention enumeration and timing side channels.

2.8 A08 — Software and Data Integrity Failures

1. Theory

Definition. Failing to verify the *integrity* of code and data: unsigned/unverified software updates, untrusted deserialization, CI/CD pipelines that pull unverified dependencies or plugins, and auto-update mechanisms without signature checks. Includes **insecure deserialization** (untrusted serialized objects executing code).

2. Visual Diagram (ASCII)

```
update/artifact/dependency → [is it signed & verified?]
  └─ NO → attacker substitutes malicious version → RCE / supply-chain compromise
  └─ YES → verify signature/hash before use (secure)
deserialization: untrusted bytes → object graph → gadget chain → code execution
```

3. Real-Life Example

A build pipeline pulls a plugin from an unpinned, unverified source; an attacker publishes a malicious version and gains code execution across every build. Or an app deserializes a user-supplied object and triggers a gadget chain leading to RCE.

4. How the Attack Happens

Substitute a malicious update/dependency (no signature check), poison the CI/CD supply chain, or send a crafted serialized payload to a deserialization endpoint.

5. How the Tester Finds It

Check that updates/artifacts are signed and verified, dependencies are pinned with integrity hashes (lockfiles), CI/CD uses trusted, pinned actions, and that the app never deserializes untrusted data with unsafe deserializers.

6. Manual Test Cases

ID	Test	Expected
INT-01	Modify an artifact, attempt deploy	Signature/hash check fails
INT-02	Endpoint accepting serialized objects	Rejects/validates; no native deser of untrusted input
INT-03	Dependency lockfile with hashes	Present and enforced
INT-04	CI actions pinned to SHAs	Yes (not floating tags)

7–13. Tools / Code

- **Tools:** `cosign` / Sigstore for signing/verification, SLSA provenance, SCA with integrity checks.
- Pin GitHub Actions to commit SHAs, not mutable tags; use `--require-hashes` (pip) / integrity in lockfiles.

14–16. CI/CD

```
# pin actions to SHA, verify artifact signature before deploy
- uses: actions/checkout@<full-40-char-sha>
- run: cosign verify-blob --key cosign.pub --signature app.sig app.tar.gz
```

17–21. Interview / Viva / Mistakes / Tips

- **Q:** What is insecure deserialization and why is it dangerous? (*Untrusted bytes reconstructed into objects can trigger code-execution gadget chains.*)
- **Scenario:** "Your app auto-updates from a URL without checking a signature." — an attacker who controls/spoofs that URL ships malware to all clients; require signed updates verified against a pinned public key.
- **Mistake:** floating dependency/action versions; deserializing untrusted input with native serializers.
- **Senior tip:** talk supply-chain: signing (Sigstore/cosign), SLSA provenance, pinned/hashed dependencies, and hardened CI/CD.

2.9 A09 — Security Logging and Monitoring Failures

1. Theory

Definition. Insufficient logging, monitoring, and alerting means attacks go undetected and un-investigable.

Symptoms: no logs for auth events/failures, logs without enough context, no centralised aggregation, no alerting, and (the flip side) logs that leak secrets/PII. You can't respond to what you can't see.

2. Visual Diagram (ASCII)

```
event (login fail, access denied, high-value action)
  → log with WHO/WHAT/WHEN/WHERE/RESULT (no secrets)
  → ship to central SIEM
  → correlate + ALERT on anomalies (e.g. 1000 failed logins)
  → human/automation responds
missing any arrow → breach detected weeks later, if ever
```

3. Real-Life Example

An attacker brute-forces accounts for days; because failed logins aren't logged or alerted, the breach is discovered only when data appears for sale. Conversely, an app that logs full request bodies leaks passwords into log files.

4. How the Attack Happens

Attackers rely on the absence of monitoring to operate slowly and quietly. They also target logs themselves — if logs store secrets, the log store becomes a treasure trove.

5. How the Tester Finds It

Perform sensitive actions and verify a correct, contextual, tamper-evident log entry is produced and centralised; verify alerts fire on abuse patterns; and verify logs **don't** contain passwords, tokens, or full PII.

6. Manual Test Cases

ID	Test	Expected
LOG-01	Failed login	Logged with actor/time/IP/result
LOG-02	Access-control denial	Logged and alertable
LOG-03	Inspect logs for secrets	No passwords/tokens/PII
LOG-04	Simulate brute force	Alert triggered

7–13. Tools / Code

- **Tools:** ELK/Splunk/SIEM, log-based alerting.

```
# after a failed login, a test hook confirms an audit entry exists and is scrubbed
requests.post(f"{BASE}/login", json={"u":"x","p":"wrong"})
entry = fetch_last_audit() # test-only accessor
assert entry["event"] == "login_failed" and entry["actor"]
assert "password" not in json.dumps(entry).lower()
```

14–16. CI/CD

Tests assert audit events are emitted for critical actions and are free of secrets; alert rules are version-controlled and tested.

17–21. Interview / Viva / Mistakes / Tips

- **Q:** What must a good security log entry contain (and never contain)? (*Who/what/when/where/result; never passwords/tokens/PII.*)
- **Scenario:** "How would you know a brute-force attack is happening in production?" — failed-login logging + rate/anomaly alerting into a SIEM with a response runbook.
- **Mistake:** logging secrets; no alerting; logs not centralised or immutable.
- **Senior tip:** tie logging to **incident response:** detection → alert → triage → contain. Mention tamper-evident, centralised logs and PII scrubbing.

2.10 A10 — Server-Side Request Forgery (SSRF)

1. Theory

Definition. SSRF tricks the *server* into making HTTP (or other) requests to a destination the attacker controls or shouldn't reach — typically internal services, cloud metadata endpoints, or localhost. It's dangerous because the server is often trusted inside the network and can reach things the attacker cannot directly.

2. Visual Diagram (ASCII)

```

attacker → app: "fetch this URL for me" (image/preview/webhook/import)
          | url = http://169.254.169.254/latest/meta-data/ (cloud metadata)
          ↓
app (inside network) → internal target → returns cloud creds / internal data
          ↓
attacker receives internal data via the app → SSRF

```

3. Real-Life Example

A "fetch image from URL" or link-preview feature accepts any URL. The attacker points it at the cloud metadata endpoint (`169.254.169.254`) and the server returns temporary cloud credentials — a full cloud-account compromise from one preview feature.

4. How the Attack Happens

Supply a URL/host the server will fetch: internal IPs (`127.0.0.1` , `10.x` , `169.254.169.254`), alternate encodings (decimal/hex IP, `[::1]` , `@` tricks, DNS rebinding), or non-HTTP schemes (`file://` , `gopher://`). Blind SSRF exfiltrates via timing or out-of-band callbacks.

5. How the Tester Finds It

Identify any feature where the server fetches a user-supplied URL (webhooks, imports, PDF/preview generators, image fetchers). Point it at internal/metadata addresses and an out-of-band listener (Burp Collaborator) to detect blind SSRF.

6. Manual Test Cases

ID	Test	Expected
SSRF-01	URL = <code>http://169.254.169.254/...</code>	Blocked
SSRF-02	URL = <code>http://127.0.0.1:8080/</code>	Blocked
SSRF-03	Encoded/ <code>@</code> /redirect bypasses	Still blocked (allow-list)
SSRF-04	<code>file:///etc/passwd</code> , <code>gopher://</code>	Scheme rejected
SSRF-05	Out-of-band callback (blind)	No server callout

7–13. API / Tools / Code

- **Burp**: use **Collaborator** to catch blind SSRF callbacks; try encoding bypasses in Repeater.

```

BAD = ["http://169.254.169.254/latest/meta-data/",
       "http://127.0.0.1:22", "file:///etc/passwd",
       "http://[::1]/", "http://2130706433/"] # decimal 127.0.0.1
for u in BAD:
    r = requests.post(f"{BASE}/api/fetch", json={"url": u}, headers=auth)
    assert r.status_code in (400,403), f"SSRF not blocked: {u}"

```

14–16. CI/CD

Regression tests assert the URL-fetch feature enforces an allow-list and blocks internal ranges/metadata each release; DAST includes SSRF checks.

17. Interview Questions

1. What is SSRF and why is the cloud metadata endpoint the classic target?
2. How do you defend against SSRF (allow-list vs blacklist)?
3. What is blind SSRF and how do you detect it?

18–19. Scenario & Expected Answer

"A profile feature fetches an avatar from a user-provided URL. How do you test and secure it?" — Test by pointing it at internal IPs, `169.254.169.254`, `localhost`, encoded IPs, and an OOB listener. Secure with a strict **allow-list** of domains/schemes, resolve-and-validate the IP (block private/link-local ranges), disable redirects/ `file://`, and isolate the fetcher (no cloud creds, network egress restricted). Senior: blacklists fail (encoding/DNS rebinding); use allow-list + network segmentation + metadata endpoint hardening (IMDSv2).

20. Common Mistakes

- Blacklisting `127.0.0.1` only (misses `:::1`, decimal IPs, DNS rebinding).
- Forgetting non-HTTP schemes and redirects.
- Ignoring blind SSRF.

21. Senior Interview Tips

Say "allow-list destinations, validate the *resolved* IP against private ranges, block redirects and non-HTTP schemes, and segment the network so the fetcher can't reach metadata/internal services." Mention IMDSv2 and Burp Collaborator for blind detection.

Section 2 recap — the Top 10 in one breath. A01 Broken Access Control (deny-by-default, server-side, per object & function) · A02 Cryptographic Failures (TLS + at-rest + hash passwords) · A03 Injection (parameterise, encode output) · A04 Insecure Design (threat model, abuse cases) · A05 Misconfiguration (hardened, minimal, IaC) · A06 Vulnerable Components (SBOM + SCA + patch) · A07 Auth Failures (MFA + throttling + strong recovery) · A08 Integrity Failures (sign & verify, pin deps, safe deser) · A09 Logging/Monitoring Failures (log + centralise + alert, no secrets) · A10 SSRF (allow-list + validate resolved IP + segment).

Section 12 — Cheat Sheets

Rapid-reference tables for the day before an interview. Skim these last.

12.1 Security Response Headers

Header	Purpose	Good value
<code>Strict-Transport-Security</code>	Force HTTPS	<code>max-age=31536000; includeSubDomains; preload</code>
<code>Content-Security-Policy</code>	Limit script/resource origins (anti-XSS)	<code>default-src 'self'; script-src 'self'</code> (nonces, no <code>'unsafe-inline'</code>)
<code>X-Content-Type-Options</code>	Stop MIME sniffing	<code>nosniff</code>
<code>X-Frame-Options</code> / CSP <code>frame-ancestors</code>	Anti-clickjacking	<code>DENY</code> / <code>frame-ancestors 'none'</code>
<code>Referrer-Policy</code>	Limit referrer leakage	<code>no-referrer</code> or <code>strict-origin-when-cross-origin</code>
<code>Permissions-Policy</code>	Restrict features	<code>geolocation=(), camera=()</code>
<code>Cache-Control</code>	Prevent caching secrets	<code>no-store</code> on sensitive responses

12.2 Key HTTP Status Codes (security lens)

Code	Meaning	Security use
200	OK	Watch for data you shouldn't see (IDOR)
301/308	Redirect	HTTP→HTTPS enforcement
400	Bad Request	Input validation rejected payload
401	Unauthorized	Not authenticated
403	Forbidden	Authenticated but not allowed (authZ)
404	Not Found	Often used to hide existence (anti-enumeration)
405	Method Not Allowed	Verb tampering check
413	Payload Too Large	Resource-exhaustion cap
429	Too Many Requests	Rate limiting working
500	Server Error	Should not leak stack traces

Remember the 401-vs-403 distinction — interviewers test it constantly. 401 = *who are you?* 403 = *you can't do that.*

12.3 Cookie Attributes

Attribute	Effect
<code>HttpOnly</code>	JS cannot read cookie (mitigates XSS token theft)
<code>Secure</code>	Sent only over HTTPS
<code>SameSite=Strict/Lax/None</code>	Cross-site sending control (CSRF mitigation)
<code>Domain / Path</code>	Scope of the cookie
<code>__Host-</code> prefix	Locks cookie to host, requires Secure + Path=/ + no Domain
<code>Max-Age / Expires</code>	Lifetime

12.4 JWT Structure & Attack Checklist

```
header.payload.signature  (each base64url)
header = {"alg":"RS256","typ":"JWT"}
payload = {"sub","iss","aud","exp","iat","nbf","role", ...}
signature = SIGN(base64(header)+"."+base64(payload), key)
```

Attack checklist: `alg:none` · RS256 → HS256 confusion · weak HMAC secret (hashcat `-m 16500`) · tampered claims (`role`) · expired `exp` not enforced · missing `aud` / `iss` checks · sensitive data in payload · JWT in `localStorage` · no revocation.

12.5 OAuth 2.0 / OIDC Quick Reference

Term	Meaning
Resource Owner	The user
Client	The app requesting access
Authorization Server	Issues tokens
Resource Server	The API
Access Token	Authorizes API calls (scopes)
ID Token (OIDC)	Proves <i>who</i> the user is (JWT)
Refresh Token	Gets new access tokens
PKCE	Protects code from interception
<code>state</code>	CSRF protection on callback
Recommended flow	Authorization Code + PKCE

12.6 SQL Injection Payloads (for authorised testing)

Goal	Payload
Auth bypass	' OR '1'='1' --
Comment out rest	admin'--
Union (column count first)	' ORDER BY 5-- then ' UNION SELECT 1,2,3--
Error-based (MySQL)	' AND extractvalue(1,concat(0x7e,version()))--
Blind boolean	' AND 1=1-- vs ' AND 1=2--
Blind time (MySQL/PG/MSSQL)	' AND SLEEP(5)-- / '; SELECT pg_sleep(5)-- / '; WAITFOR DELAY '0:0:5'--
Stacked	'; DROP TABLE users--
NoSQL (Mongo)	{"\$ne": null} / {"\$gt": ""}

12.7 XSS Payloads (for authorised testing)

Context	Payload
Basic	<script>alert(1)</script>
Attribute break-out	"><script>alert(1)</script>
Event handler	
SVG	<svg onload=alert(1)>
JS context	';alert(1)//
Filter evasion	Mixed case <sCrIpT> , HTML entities, no-quote handlers
DOM sink test	# in fragment

12.8 Essential curl Commands

```

curl -i https://t.example/api/health # headers + body
curl -H "Authorization: Bearer $TOKEN" https://t.example/api/me
curl -X POST -H 'Content-Type: application/json' \
  -d '{"q":"test"}' https://t.example/api/search
curl -k https://t.example # ignore TLS (testing only)
curl -x http://127.0.0.1:8080 https://t.example # route via Burp proxy
curl -sD - -o /dev/null https://t.example # headers only
curl -H "Origin: https://evil.example" -i https://t.example/api # CORS probe

```

12.9 Essential Linux / Recon Commands

```
nmap -sV -p- target.example      # port/service discovery
whatweb https://target.example   # tech fingerprint
gobuster dir -u https://t.example -w wordlist.txt # content discovery
nikto -h https://target.example  # misconfig scan
testssl.sh target.example        # TLS grade
gitleaks detect --no-git -v      # secret scan
jwt_tool <token> -T              # JWT tamper toolkit
sqlmap -u 'https://t/x?id=1' --batch # SQLi (authorised only)
```

12.10 Encryption / Hashing Comparison

Type	Examples	Reversible?	Use for
Encoding	Base64, URL, hex	Yes (no secret)	Transport, not security
Symmetric encryption	AES-GCM, ChaCha20-Poly1305	Yes (shared key)	Data you must read back
Asymmetric encryption	RSA, ECC	Yes (key pair)	Key exchange, signatures
Hashing (general)	SHA-256, SHA-3	No	Integrity, checksums
Password hashing (KDF)	Argon2id, bcrypt, scrypt	No (slow, salted)	Passwords only

12.11 Burp Suite Quick Map

Tool	Use
Proxy	Intercept/modify requests
Repeater	Manual re-send & tamper (IDOR, JWT, injection)
Intruder	Fuzz/brute (payload positions)
Sequencer	Token randomness/entropy
Decoder	Encode/decode base64/URL/hex
Comparer	Diff responses
Collaborator	Out-of-band (blind SSRF/XXE)
Extensions	JWT Editor, Authorize (authZ), Param Miner

Section 14 — Final Interview Preparation & Revision Plans

14.1 One-Day Revision (the night before)

1. **CIA triad** as a severity lens; **AAA**; horizontal vs vertical escalation.
2. **Encryption vs hashing vs encoding** (never confuse these).
3. **JWT attack list** (**alg:none**, RS256 → HS256, weak secret, tampered claims, expired, missing aud/iss).
4. **OAuth vs OIDC** one-liner; Authorization Code + PKCE; state.
5. **CORS (reads) vs CSRF (writes)**; SameSite; anti-CSRF tokens.
6. **OWASP Top 10** — name all ten + one defence each (use the Section 2 recap).
7. **401 vs 403**; must-have security headers; CSP.
8. **IDOR/BOLA test method** (two users, swap IDs).
9. **SQLi defence** = parameterised queries; **XSS defence** = output encoding + CSP.
10. **SSRF** = allow-list + validate resolved IP + segment; metadata endpoint.

14.2 Three-Day Revision

- **Day 1:** Section 1 fundamentals end-to-end; write the JWT and OAuth diagrams from memory.
- **Day 2:** Section 2 OWASP Top 10; for each, be able to say *definition* → *how you'd test* → *the fix*.
- **Day 3:** Cheat sheets + do one lab (Juice Shop) hands-on; practise explaining findings out loud in the "impact → evidence → fix" format.

14.3 Seven-Day Revision

Day	Focus
1	CIA, AAA, encryption/hashing, TLS/PKI
2	Sessions/cookies/JWT/OAuth/OIDC
3	CORS/CSRF/SOP, headers/CSP, rate limiting/WAF/secrets
4	OWASP A01–A05 with test methods
5	OWASP A06–A10 with test methods
6	Tools: Burp/ZAP/Postman workflows + cheat sheets
7	Mock viva out loud; a lab; bug-report writing

14.4 Thirty-Day Plan (fresher → interview-ready)

- **Week 1 — Fundamentals:** work through all of Section 1; reproduce every ASCII diagram from memory; set up Burp + Postman + a Python `requests` harness.
- **Week 2 — OWASP + tools:** Section 2 in depth; run ZAP/Burp against DVWA and Juice Shop; find one instance of each Top-10 class in a lab.
- **Week 3 — API & automation:** build a small pytest/RestAssured security suite (authZ matrix, JWT tampering, injection payloads, rate-limit check); wire it into GitHub Actions.
- **Week 4 — Interview polish:** practise scenario viva answers using the "impact → how attacker → how tester → fix" structure; write two full security bug reports; do three timed mock interviews.

14.5 The Answer Framework (use for every question)

Definition → Why it matters (impact) → How the attack happens → How I'd test it → How to fix it → Defence-in-depth.

Delivering answers in this order instantly signals seniority. Even for a term you half-remember, walking these five beats sounds structured and competent.

14.6 Top interview questions — rapid self-test

1. Explain the CIA triad with an example per leg.
 2. Authentication vs authorization; 401 vs 403.
 3. Horizontal vs vertical privilege escalation; what is IDOR/BOLA?
 4. Everything you'd test on a login API returning a JWT.
 5. `alg:none` and RS256 → HS256 attacks.
 6. OAuth vs OIDC; why PKCE and `state`.
 7. Does CORS prevent CSRF? Why not?
 8. Best defence against SQL injection; blind vs union vs error-based.
 9. Stored vs reflected vs DOM XSS; best defences.
 10. What is SSRF; why is the metadata endpoint the classic target; how to defend.
 11. Why hash passwords instead of encrypting; which algorithm.
 12. Name the OWASP Top 10 and one mitigation each.
 13. How do you find vulnerable dependencies (SBOM/SCA)?
 14. What makes a secure password-reset flow?
 15. What must (and must not) a security log entry contain?
-

Appendix A — Glossary

AAA — Authentication, Authorization, Accounting. **ABAC** — Attribute-Based Access Control. **AEAD** — Authenticated Encryption with Associated Data (e.g. AES-GCM). **BFLA** — Broken Function-Level Authorization. **BOLA** — Broken Object-Level Authorization (a.k.a. IDOR). **CA** — Certificate Authority. **CIA** — Confidentiality, Integrity, Availability. **CORS** — Cross-Origin Resource Sharing. **CSP** — Content Security Policy. **CSRF** — Cross-Site Request Forgery. **CVE** — Common Vulnerabilities and Exposures. **CVSS** — Common Vulnerability Scoring System. **DAST** — Dynamic Application Security Testing. **HSTS** — HTTP Strict Transport Security. **IDOR** — Insecure Direct Object Reference. **JWT** — JSON Web Token. **KDF** — Key Derivation Function (Argon2/bcrypt/scrypt). **KMS/HSM** — Key Management Service / Hardware Security Module. **MFA** — Multi-Factor Authentication. **OIDC** — OpenID Connect. **PKCE** — Proof Key for Code Exchange. **PKI** — Public Key Infrastructure. **RBAC** — Role-Based Access Control. **SAST** — Static Application Security Testing. **SBOM** — Software Bill of Materials. **SCA** — Software Composition Analysis. **SIEM** — Security Information and Event Management. **SOP** — Same-Origin Policy. **SSRF** — Server-Side Request Forgery. **STRIDE** — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. **TLS/SSL** — Transport Layer Security / (deprecated) Secure Sockets Layer. **WAF** — Web Application Firewall. **XSS** — Cross-Site Scripting. **XXE** — XML External Entity.

Appendix B — HTTP Status Codes Reference (abridged)

1xx informational. **2xx** success (200 OK, 201 Created, 204 No Content). **3xx** redirection (301/302/308). **4xx** client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 409 Conflict, 413 Payload Too Large, 415 Unsupported Media Type, 429 Too Many Requests). **5xx** server error (500 Internal, 502 Bad Gateway, 503 Unavailable, 504 Timeout).

Appendix C — References & Further Reading

- OWASP Top 10 (2021) and the OWASP Testing Guide (WSTG).
- OWASP API Security Top 10.
- OWASP Cheat Sheet Series (Authentication, Session Management, CSRF, XSS, JWT, Password Storage).
- PortSwigger Web Security Academy (free labs).
- NIST SP 800-63B (Digital Identity Guidelines).
- The MITRE ATT&CK and CWE databases.
- RFCs: 6749 (OAuth 2.0), 7519 (JWT), 6797 (HSTS), 7617/6265 (cookies), 6454 (origin).

Appendix D — Index (selected)

Access control 2.1 · AAA 1.2 · Argon2/bcrypt 1.4, 2.2 · Authentication failures 2.7 · CIA triad 1.1 · Cookies 1.7, 12.3 · CORS 1.10, 12.8 · CSP 1.11, 12.1 · CSRF 1.10 · Cryptographic failures 2.2 · Digital signature 1.6 · Encryption 1.3 · Headers 1.11, 12.1 · Injection 2.3 · Insecure design 2.4 · JWT 1.8, 2.7, 12.4 · Logging failures 2.9 · Misconfiguration 2.5 · OAuth/OIDC 1.9, 12.5 · PKI 1.6 · Rate limiting 1.12 · Secrets/rotation 1.13 · Sessions 1.7 · SSRF 2.10 · TLS/SSL 1.5 · Vulnerable components 2.6 · WAF 1.12 · XSS 2.3, 12.7.

End of Volume 1

This volume delivered, complete and without placeholders: the front matter and usage guide; **Section 1 — Security Fundamentals** (13 topics, full 21-part format on the core four and complete coverage of the rest); **Section 2 — OWASP Top 10** (all ten in the 21-part format); the **cheat-sheet pack** (Section 12); and the **revision plans + interview framework** (Section 14) with glossary, HTTP-code reference, references, and index.

Volumes still to be built (each will be delivered complete in the same style, on request): Section 3 API Security deep-dive · Section 4 the 250+ scenario-based API viva bank · Section 5 Web Security Testing · Section 6 Database Security · Section 7 Mobile App Security · Section 8 Automation & CI/CD Security · Section 9 Security Bug-Report Writing · Section 10 Company-style interview banks · Section 11 Mock Viva (Beginner/Intermediate/Advanced) · Section 13 Hands-on Labs.